

Constantin Shoot Vayevoda

Development of a Contextual Sensing Kit to
Assess the Relationship Between Air
Quality and Noise

Treball de Fi de Grau

dirigit per Edgar Batista de Frutos
i Antoni Martínez Ballesté

Grau en Enginyeria Biomèdica



UNIVERSITAT ROVIRA i VIRGILI

Tarragona

2025

Abstract

This project, the first phase of a two-part research effort within a smart classroom framework, develops and validates two modular IoT sensing kits for environmental monitoring: *AirQ* kit for air quality and *Noise* kit for acoustic analysis. The objective is to create robust platforms while collecting contextual-environment data to explore correlations between air quality metrics and human voice-band activity. *AirQ* kit integrates CO₂, particulate matter, temperature, humidity, and light detection with local microSD logging and Wi-Fi-based time synchronization, running on Python. *Noise* kit performs high-frequency audio acquisition at 44.1 kHz, applies short-time Fourier transform analysis to detect spectral activity in the voice range, logs results locally and sends peak data via BLE when the pushbutton is pressed. It is implemented in C/C++ on the ESP32 Arduino core, utilizing ESP-IDF's RTOS features for precise timing, multitasking, and energy efficiency. Custom-designed 3D-printed enclosures provide robust, compact builds with carefully optimized sensor placement and allow rapid assembly, maintenance, and component replacement. Contextual environment testing confirmed autonomous operation, reliable data acquisition, and provided evidence that noise metrics can serve as proxies for variations in indoor CO₂, thereby establishing a foundation for integration into a distributed, self-organizing monitoring network in Phase 2.

Keywords: Smart environments, monitoring unit, Internet of Things, RTOS, ESP32, Raspberry Pi, FFT-based acoustic monitoring

Resum

Aquest projecte, primera fase d'un esforç de recerca dividit en dues parts dins d'un marc d'aula intel·ligent, desenvolupa i valida dos kits de sensat modulables IoT per al monitoratge ambiental: el kit *AirQ* per a la qualitat de l'aire i el kit *Noise* per a l'anàlisi acústica. L'objectiu és crear plataformes robustes mentre es recopilen dades contextuais de l'entorn per explorar correlacions entre mètriques de qualitat de l'aire i l'activitat en la banda de la veu humana. El kit *AirQ* integra detecció de CO₂, material particulat, temperatura, humitat i llum, amb registre local en microSD i sincronització horària via Wi-Fi, funcionant en Python. El kit *Noise* realitza adquisició d'àudio d'alta freqüència a 44,1kHz, aplica anàlisi de transformada de Fourier de curt termini per detectar activitat espectral en el rang vocal, registra els resultats localment i envia les dades de pics via BLE quan es prem el botó. Està implementat en C/C++ sobre el nucli Arduino de l'ESP32, utilitzant les funcions RTOS d'ESP-IDF per aconseguir temporització precisa, multitarea i eficiència energètica. Les carcasses dissenyades a mida i impreses en 3D proporcionen construccions compactes i robustes, amb una col·locació de sensors acuradament optimitzada i permeten un muntatge, manteniment i substitució de components ràpids. Les proves en entorns contextuais van confirmar l'operació autònoma, l'adquisició fiable de dades i van aportar evidència que les mètriques acústiques poden servir com a indicadors de les variacions de CO₂ en interiors, establint així una base per a la integració en una xarxa de monitoratge distribuïda i autoorganitzada a la Fase 2.

Paraules clau: Entorns intel·ligents, unitat de monitoratge, Internet de les Coses, RTOS, ESP32, Raspberry Pi, monitoratge acústic basat en FFT

Resumen

Este proyecto, primera fase de un esfuerzo de investigación dividido en dos partes dentro de un marco de aula inteligente, desarrolla y valida dos kits de sensorado modulares IoT para la monitorización ambiental: el kit *AirQ* para calidad del aire y el kit *Noise* para análisis acústico. El objetivo es crear plataformas robustas mientras se recopilan datos contextuales del entorno para explorar correlaciones entre métricas de calidad del aire y la actividad en la banda de la voz humana. El kit *AirQ* integra detección de CO₂, material particulado, temperatura, humedad y luz, con registro local en microSD y sincronización horaria vía Wi-Fi, funcionando en Python. El kit *Noise* realiza adquisición de audio de alta frecuencia a 44,1kHz, aplica análisis de transformada de Fourier de corto plazo para detectar actividad espectral en el rango vocal, registra los resultados localmente y envía los datos de picos vía BLE cuando se pulsa el botón. Está implementado en C/C++ sobre el núcleo Arduino del ESP32, utilizando las funciones RTOS de ESP-IDF para lograr temporización precisa, multitarea y eficiencia energética. Carcasas diseñadas a medida e impresas en 3D proporcionan construcciones compactas y robustas, con una colocación de sensores cuidadosamente optimizada y permiten un montaje, mantenimiento y sustitución de componentes rápidos. Las pruebas en entornos contextuales confirmaron la operación autónoma, la adquisición fiable de datos y aportaron evidencia de que las métricas acústicas pueden servir como indicadores de las variaciones de CO₂ en interiores, estableciendo así una base para la integración en una red de monitorización distribuida y autoorganizada en la Fase 2.

Palabras clave: Entornos inteligentes, unidad de monitorización, Internet de las Cosas, RTOS, ESP32, Raspberry Pi, monitorización acústica basada en FFT

Acknowledgements

I would like to express my sincere gratitude to my supervisors, Dr. Antoni Martínez and Dr. Edgar Batista, for offering me the opportunity to collaborate on their project, for trusting in my capabilities, and for granting me the freedom and flexibility to self-organize my work. Their guidance, encouragement, and confidence have been invaluable to both my professional and personal growth. I am deeply thankful to my family and to my partner for their unwavering support, patience, and motivation throughout this journey, which have been essential in bringing this work to completion.

Contents

List of Acronyms	xi
1 Introduction	1
1.1 Motivation	1
1.2 Problem	1
1.3 Objectives	1
1.4 Scope and limitations	2
2 Background	3
2.1 ESP32 Platform and RTOS Foundations	3
2.2 Fourier Analysis and Speech Spectrum	3
2.3 3D Printing Basics	4
2.4 Data Features and Processing	4
3 Methods	5
3.1 Overview of the two kits	5
3.2 AirQ	6
3.3 Noise	14
3.4 Data Preparation and Analysis Software	24
4 Validation and Baseline	27
4.1 AirQ readiness	27
4.2 Noise fallbacks	27
4.3 Cycle metrics (Noise kit)	28
4.4 BLE peak path (Noise kit)	28
4.5 Battery autonomy (Noise kit)	28
4.6 Mechanical robustness	29
4.7 Analysis-software validation	29
5 Experimental Setup	30
5.1 Site	30
5.2 Layout/positions	30
5.3 Schedule	31
5.4 Procedures	32
6 Results and Discussion	32
6.1 Decision rule and hypotheses	32
6.2 Why emphasise the presence-controlled (occupied residuals) scope? . .	33
6.3 Pair-level summaries at a glance	33
6.4 Layout-aware reading of the observed results	34
6.5 Hypothesis decisions (per pair, per scope)	34
6.6 Immediate interpretation of the gathered data	35
6.7 Operational notes on the kits	36
6.8 Illustrative figure: NOISE102 voice timeline	36
7 Future Work	37

8	Conclusions	38
A	Enclosure Prototyping and 3D-Print Iterations	40
B	Budget and Procurement Summary	44
	References	45

List of Figures

Figure 1. Schematic “speech banana”: vowel and consonant energy across frequency and level [18].	4
Figure 2. Final form factor and produced units.	6
Figure 3. Reference hardware from the ACTUA project used as the starting point for AirQ [24].	6
Figure 4. AirQ logical schematic and wiring [31, 32, 33, 34].	8
Figure 5. Early AirQ fully wired prototype.	9
Figure 6. Two views of the late AirQ prototype.	9
Figure 7. General view of the AirQ enclosure parts (top and bottom). . . .	10
Figure 8. Transparent view of the AirQ enclosure showing 3D models of components.	11
Figure 9. AirQ enclosure design: (a) bayonet socket concept, (b) finished socket design, and (c) multi-view highlighting interior details. . .	12
Figure 10. Preparation of the two enclosure halves in Bambu Studio before printing.	13
Figure 11. Noise kit logical schematic and wiring [55].	16
Figure 12. Noise kit prototyping stages: (a) early prototype using a bread-board for bring-up and (b) late-stage prototype without enclosure. .	17
Figure 13. General view of the Noise enclosure parts (top and bottom). . . .	18
Figure 14. Transparent view of the Noise enclosure showing component placement and clearances.	19
Figure 15. Noise enclosure details: (a) interior features; (b) pushbutton perf-board assembly and rail-stopper sliding mechanism.	20
Figure 16. Preparation of the two enclosure halves in Bambu Studio before printing.	20
Figure 17. Frame plotted from a microSD binary log.	23
Figure 18. Frame received via the WebSocket tool.	23
Figure 19. Noise kit autonomy with twin axes: battery percentage (left) and voltage (right) versus time since start (hours). Annotations mark the initial state (100 %, 4.20 V) and the stop condition (< 5 %, 3.63 V).	29
Figure 20. Field deployment: device placements (tags as in Table 8).	31
Figure 21. Commissioning overview: full set of kits and Bluetooth verification screenshot.	31

Figure 22.	Voice activity timeline — NOISE102. Blue: <code>voice_rate_time</code> . Orange: 5-minute rolling mean. Dashed lines: reference thresholds. Right axis: mean-rise intensity (dB proxy). The figure illustrates the aggregation and processing pipeline that yields the minute-scale voice metrics used in the analysis.	36
Figure 23.	AirQ slightly modified base model, first verification print. <i>Left</i> : side view; <i>right</i> : front view. The Raspberry Pi fit confirmed that the original model required correction to align the USB-C charging port with the lower half. The print also validated dimensional correspondence between the Raspberry Pi 3D model and the real hardware, clearing it for continued use in shaping the enclosure. .	40
Figure 24.	AirQ enclosure bottom halves—final production. <i>Left</i> : bottom parts being printed; <i>right</i> : completed batch of three bottom parts.	41
Figure 25.	Noise kit enclosure—top-piece production. <i>Left</i> : live print on a Bambu Lab P1S; <i>right</i> : close-up after completion, showing surface quality and dimensional features.	42
Figure 26.	Form-factor comparisons. <i>Left</i> : Noise and AirQ together (antenna slit and microphone aperture on the Noise enclosure; microSD slit on the AirQ enclosure); <i>right</i> : AirQ beside an ACTUA project kit, highlighting the size difference and the grille-inspired aesthetic.	42
Figure 27.	Production overview and iterations. <i>Left</i> : the complete set of fabricated units; <i>right</i> : representative failed/test pieces illustrating the trial-and-error refinements that led to the definitive enclosure parts for both kits.	43

List of Tables

Table 1. AirQ core components.	7
Table 2. AirQ connectivity.	7
Table 3. AirQ measurements mapped to sensors.	10
Table 4. Summary of printing details for each half of the AirQ enclosure (batch of three units).	13
Table 5. Noise kit core components.	15
Table 6. Noise kit connectivity.	16
Table 7. Summary of printing details for each half of the Noise enclosure (batch of three units).	20
Table 8. Devices, power mode, placement, and inclusion.	30
Table 9. AIRQ001 $\times$ NOISE102 — best-lag summary across scopes (ex- perimental dataset). PRESENCE-CTRL is the <i>primary</i> scope for short-lag inference.	33
Table 10. AIRQ002 $\times$ NOISE102 — best-lag summary across scopes (ex- perimental dataset). PRESENCE-CTRL is the <i>primary</i> scope for short-lag inference.	34
Table 11. Purchased components (excluding recycled parts). Column widths and alignment are adjusted to avoid overlap and place the second table flush right.	44
Table 12. Overall expenditure for purchased items.	44

List of Acronyms

ADC	Analog-to-Digital Converter
API	Application Programming Interface
BH–FDR	Benjamini–Hochberg False Discovery Rate (multiple-testing control)
BLE	Bluetooth Low Energy
CAD	Computer-Aided Design
CI	Confidence Interval
CSV	Comma-Separated Values
DB	Database
DFT	Discrete Fourier Transform
DMA	Direct Memory Access
DSP	Digital Signal Processing
DST	Daylight Saving Time
FFT	Fast Fourier Transform
GPIO	General-Purpose Input/Output
G-code	Numerical control program for 3D printers
HAT	Hardware Attached on Top (Raspberry Pi add-on board)
HUM	Relative Humidity (context variable)
HVAC	Heating, Ventilation and Air Conditioning
ICT	Information and Communication Technology
IEQ	Indoor Environmental Quality
I²C	Inter-Integrated Circuit serial bus
I²S	Integrated Interchip Sound serial bus
ISR	Interrupt Service Routine
JSON	JavaScript Object Notation
LCD	Liquid-Crystal Display
MCU	Microcontroller Unit
MiB/GiB	Mebibyte/Gibibyte (binary units, 2^{20} , 2^{30} bytes)
NTP	Network Time Protocol

NVS	Non-Volatile Storage (ESP32 flash key–value store)
PCB	Printed Circuit Board
PLA	Polylactic Acid (3D-print filament)
PM	Particulate Matter (mass concentration)
PM₁	PM with aerodynamic diameter $\leq 1\text{ }\mu\text{m}$
PM_{2.5}	PM with aerodynamic diameter $\leq 2.5\text{ }\mu\text{m}$
PM₁₀	PM with aerodynamic diameter $\leq 10\text{ }\mu\text{m}$
PSRAM	Pseudo-Static Random-Access Memory (external RAM)
QA	Quality Assurance (sanity/health checks)
RMS	Root-Mean-Square
RTC	Real-Time Clock
RTOS	Real-Time Operating System
SAR	Successive Approximation Register (ADC architecture)
SFM	Spectral Flatness Measure
SMP	Symmetric Multiprocessing
SNR	Signal-to-Noise Ratio
SNTP	Simple Network Time Protocol (ESP-IDF binding)
SoC	System-on-Chip
SPI	Serial Peripheral Interface
SQL/SQLite	Structured Query Language / Embedded SQL database engine
SRAM	Static Random-Access Memory
STFT	Short-Time Fourier Transform
TEMP	Air Temperature (context variable)
TFT	Thin-Film Transistor (LCD subtype)
UART	Universal Asynchronous Receiver/Transmitter
UI	User Interface
UTC	Coordinated Universal Time
ΔCO_2	Short-horizon change in CO_2 (e.g., 3-min mean derivative)
YAML	YAML Ain’t Markup Language (configuration format)

1 Introduction

1.1 Motivation

Indoor environmental quality (IEQ)—particularly air and acoustic conditions—has a direct impact on student comfort and short-term learning outcomes in classrooms [1]. Smart-environment systems address these needs by integrating heterogeneous sensor streams to infer context and support action, often spanning edge, fog, and cloud layers [2]. Application domains include ambient-assisted living [3], smart classrooms [4], and smart cities [2]. Yet real deployments remain challenging due to device heterogeneity, constrained resources, and interoperability gaps [5, 2]. Anticipating multi-unit, distributed deployments, self-organisation and automatic configuration are required to avoid fragile, manual setups [6, 7].

In this context, compact, modular kits that jointly monitor air quality and voice-band activity—while producing synchronised, privacy-preserving evidence—offer a practical foundation for credible IEQ studies. This work therefore focuses on *Phase 1*: robust on-device sensing with accurate timestamping, establishing a reliable baseline for later networking and distributed coordination.

1.2 Problem

Many classroom-oriented sensing solutions are ad hoc and difficult to compare across sites: they mix sensors and protocols, require manual setup, and often lack robust timestamping or reproducible data retention [2, 5]. The result is unsynchronised streams, incomplete metadata, and weak reproducibility, which hinder scale-out.

For IEQ studies, the essential evidence is the *simultaneous, synchronised* measurement of key air-quality variables (e.g., CO₂, particulate matter, temperature, humidity) together with *non-intelligible* indicators of voice-band activity, all retained with timestamps and metadata [1]. To protect privacy, raw speech is not recorded. To ensure reliability, systems must function independently of external networks or fluctuating power. For future multi-unit use, they should also reduce manual setup by supporting self-organisation and clear module roles, since static configurations do not scale [6, 7].

Phase 1 boundary. This foundational gap is addressed by designing two low-cost, modular kits that perform *synchronised, timestamped on-device acquisition* (air quality and voice-band indicators) with durable data retention and deployable enclosures. Networking, dashboards, and distributed coordination are *intentionally deferred to Phase 2*.

1.3 Objectives

The objectives of Phase 1 are deliberately scoped to ensure robust operation while laying the groundwork for future expansion:

1. Reuse and adapt components from the *ACTUA* sensor kit [7] as a proven starting point.
2. Engineer a modular and efficient architecture with clear interfaces, simplifying future Phase 2 extensions.

3. Develop two sensing kits: *AirQ* (air-quality metrics) and *Noise* (voice-band activity indicators).
4. Achieve autonomous acquisition with accurate, timestamped on-device retention.
5. Deliver compact, serviceable 3D-printed enclosures for indoor deployment.
6. Integrate only lightweight IoT hooks in Phase 1, consistent with the task framing.
7. Validate the kits in a contextual indoor environment.
8. Provide an initial analysis of correlations between air-quality metrics and voice-band activity using the collected data.

1.4 Scope and limitations

This Phase 1 work delivers two modular kits (*AirQ*, *Noise*) with timestamped on-device retention, printable enclosures, and validation through a single contextual deployment. The main constraints are:

- IoT functionality is restricted to lightweight hooks; full networking, dashboards, and cloud integration are deferred to Phase 2.
- Commodity sensors with basic calibration are used; measurements are not regulatory-grade.
- Privacy is preserved: no raw speech is stored—only derived, non-intelligible indicators.
- Findings are site- and setup-specific from limited data; not directly generalisable.
- Prototypes are not industrially certified; long-term robustness and compliance remain out of scope.

Document structure. The remainder of this document is organised as follows. **Background** reviews the technological, analytical, and methodological foundations that frame the project and explain the basis for the design choices. **Methods** presents the two kits—hardware, enclosures, firmware—and the data preparation and analysis pipeline at a level sufficient for reproduction. **Validation and Baseline** reports pre-deployment checks and establishes operating baselines. **Experimental Setup** details the deployment context, device placement, schedule, and procedures. **Results and Discussion** present the analysed outcomes, relate them to the study aims, and note limitations and practical implications. **Future Work** outlines the roadmap for Phase 2, and **Conclusions** summarise the main contributions and takeaways. Finally, **References** and **Appendices** compile the cited sources and supplementary materials (additional figures, budget breakdown, and implementation details).

2 Background

2.1 ESP32 Platform and RTOS Foundations

The **ESP32-S3** is a dual-core Wi-Fi/BLE SoC with rich peripherals and optional PSRAM, well suited to real-time sensing and signal processing [8]. Its SDK, **ESP-IDF**, embeds **FreeRTOS** and exposes tasks, priorities, queues, semaphores, timers, and event groups for deterministic scheduling rather than best-effort time sharing [9, 10]. Espressif’s port provides symmetric multi-processing across the two cores and system APIs; correct priority design and bounded critical sections remain essential to avoid starvation and *priority inversion* [11]. For rapid prototyping, the **Arduino-ESP32** core layers an Arduino API over ESP-IDF while keeping access to low-level drivers (Wi-Fi, BLE, ADC, SPI, I²C, PSRAM) [12]. External **PSRAM** integrates into the heap to host large buffers and arrays [13].

Scheduler and task model. FreeRTOS on ESP32 uses a *fixed-priority, preemptive* scheduler with optional time slicing among equal priorities. Tasks have independent stacks and can be *pinned* to a specific core or left for SMP placement. ISRs should be brief and *defer work* via from-ISR queue/semaphore APIs; software timers coordinate periodic actions [9, 10].

I/O and sampling. High-speed peripherals commonly use **SPI** (displays, storage); low-power sensors use **I²C**; **UART** supports debugging and external modules; **I²S** transports continuous digital audio and connects ADCs or microphones; and **GPIO** pins provide flexible digital input/output. For high-rate capture with the on-chip SAR ADC, ESP-IDF provides *continuous* mode with **DMA**, moving samples directly into memory to sustain throughput with minimal CPU intervention [14].

2.2 Fourier Analysis and Speech Spectrum

Time-domain audio is difficult to interpret directly; frequency-domain analysis reveals its spectral composition. The **Discrete Fourier Transform (DFT)** of a length- N sequence $x[n]$ is

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, \dots, N-1, \quad (1)$$

where $X[k]$ is the complex amplitude at frequency bin k [15, 16]. Direct computation requires $O(N^2)$ operations; the **FFT** reduces this to $O(N \log N)$, enabling real-time spectral analysis on embedded platforms [15].

Windowing and leakage. Finite-length frames implicitly assume periodicity, which causes *spectral leakage*. Multiplying by a smooth window (e.g., Hann or Hamming) reduces leakage and yields cleaner magnitude spectra, at the cost of wider main lobes [17].

Speech band. Human voice production spans a broad frequency range, with fundamental tones starting below a few hundred hertz and higher-frequency formants and consonant cues carrying much of the intelligibility. Fant’s plots of Swedish vowels and consonants revealed clustered formant regions, later stylised as the audiogram “*speech banana*” [18]. The overall speech spectrum extends roughly from 70 Hz to 10 kHz, with most intelligibility concentrated below 4 kHz (Fig. 1).

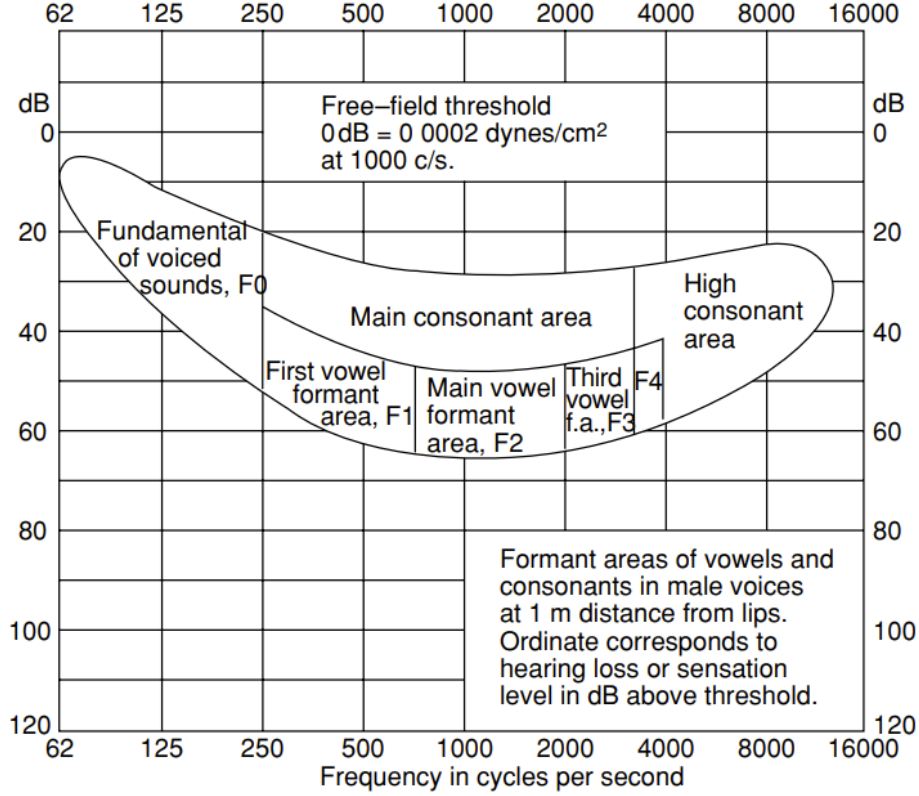


Fig. 46. Schematized presentation of the average speech spectrum within an audiogram in terms of main formant areas. Sensation levels (down) are relative to the standardized free-field threshold.

Figure 1. Schematic “speech banana”: vowel and consonant energy across frequency and level [18].

2.3 3D Printing Basics

Digital models in **STL** or **3MF** describe object geometry. A slicer program converts these models into **G-code**, which specifies layer-by-layer motion, extrusion, and temperature control; printers execute G-code, not STL/3MF directly [19]. For enclosure prototyping, **PLA** is popular due to easy printing, low warping, and good dimensional accuracy. It softens at moderate temperatures and can embrittle or discolor under prolonged sunlight, but is typically sufficient for indoor use [20].

2.4 Data Features and Processing

Quantitative signals can be characterised with compact descriptors such as **RMS level** (average signal strength), band-limited **SNR** (signal relative to background), and **spectral flatness** (**SFM**, distinguishing tone-like from noise-like spectra) [15].

Relationships between variables may be examined using **Pearson** correlation for linear dependence and **Spearman** correlation for monotonic association [21]. To account for temporal offsets, associations can be evaluated across multiple *lags*, which may reveal leading or delayed relationships. Statistical significance is typically expressed via *p*-values; when many lags or endpoints are tested, the false discovery rate can be controlled with the **Benjamini–Hochberg** procedure, reporting adjusted *q*-values [22].

Longer-term drift and periodic structure can create spurious or inflated correlations if unaddressed. Common remedies include exponential smoothing to reduce slow trends

and **cosinor** (harmonic regression with sine/cosine terms) to represent regular cycles such as daily rhythms [23].

Purpose. The Background introduces the technological and analytical foundations that frame the project, from sensing and signal analysis to fabrication and data interpretation.

3 Methods

3.1 Overview of the two kits

AirQ (from ACTUA baseline)

- The legacy ACTUA kit was *disassembled* and its firmware *reviewed end-to-end* to recover the original behaviour and coding patterns.
- Non-essential features (data sending and remote control) were *removed*; the code-base was *extended* with an additional sensor while keeping the modular structure intact.
- Hardware was *rearranged* to be more compact and maintenance-friendly, explicitly handling self-heating bias and shorting risk (cable routing, separation, strain relief).
- A *two-piece printed enclosure* was designed around the new layout for quick assembly/disassembly and repeatable mounting.
- Current behaviour: *autostart on power*, *time synchronisation over Wi-Fi* at boot, and *continuous local recording* of air-quality measurements.

Noise (from scratch)

- Only the *microphone module* from the ACTUA kit was reused; all other components and interconnects were newly selected and integrated.
- Firmware was developed *from zero* using an embedded, RTOS-based modular approach.
- On *pushbutton* press, the compact display shows *battery status*; in the same action, the *dominant frequency bin* is sent via BLE.
- The unit is *battery-powered* and includes *basic protective mechanisms* appropriate to low-level embedded operation.
- Current behaviour: *autostart on power*, *time synchronisation over Wi-Fi* at boot, and *continuous local recording* analogous to AirQ.

Kits produced

- The two kits are shown in Figure 2(a), both assembled and opened to reveal the internal layout.
- A total of *three copies* of each kit were produced (Figure 2(b)).
- *Two* copies per kit are reserved for field testing; *one* copy per kit is retained as a lab/reference unit for prototyping and quick checks.

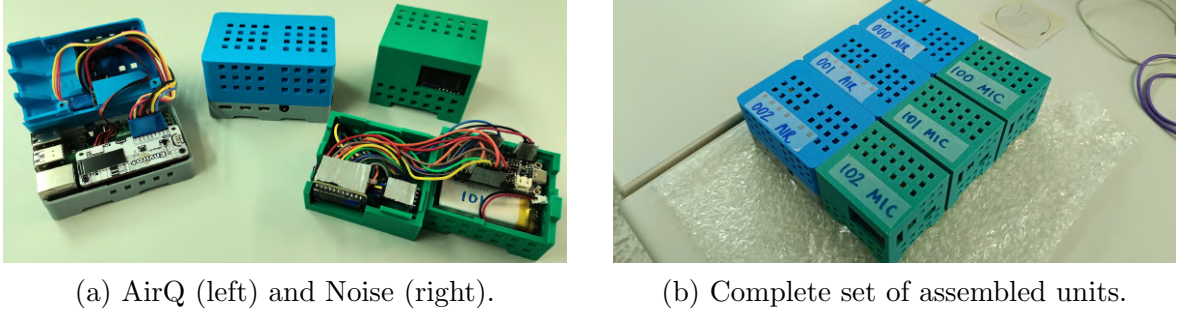


Figure 2. Final form factor and produced units.

Structure of what follows. The next sections describe each kit in turn, covering *hardware*, *3D enclosure design*, and *software/firmware*, with concise explanations focused on reproducibility.

3.2 AirQ

Hardware The **AirQ** unit is a compact redesign of an earlier classroom-monitoring kit, intended as a foundation for *decentralising* heterogeneous measurement nodes and enabling a scalable, autonomous sensor network in later stages. Most of its hardware is derived from the ACTUA platform (Figure 3).

Legacy reuse and redesign. The legacy unit was disassembled and inspected to understand interconnects, placement constraints, and component choices. Wherever possible, parts and harnesses were reused; the layout was reworked for more compact packaging, clearer cable routing, and quick access to sensing elements.

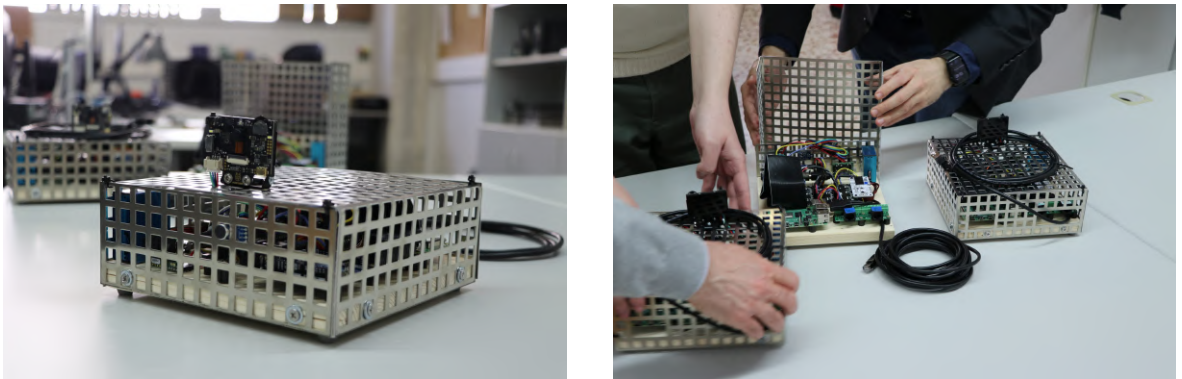


Figure 3. Reference hardware from the ACTUA project used as the starting point for AirQ [24].

Component selection. As summarised in Table 1, the only hardware added beyond the ACTUA baseline is the DHT22. In contrast, several subsystems were deliberately removed: the vision module, microphone, external light sensor, the Arduino UNO co-processor, RJ45 connectors, and the large port expander. This reconfiguration makes the kit’s purpose explicit: a focused, reliable air-quality monitor rather than a general “all-in-one” classroom node.

Component	Role	Reused	Notes
Raspberry Pi 4 Model B [25]	Controller	Yes	Onboard microSD; 40-pin header for I/O
Pimoroni Enviro + Air Quality [26, 27]	Sensor HAT	Yes	Stacks on 40-pin header; also acts as a sensor hub
Plantower PMS5003 [28]	Particulate sensor	Yes	UART interface
DHT22 [29]	Temp./Hum.	No	Single-wire GPIO data
Sensirion SCD30 [30]	CO ₂ sensor	Yes	I ² C wiring
microSD	Storage	Yes	System, code, and database

Table 1. AirQ core components.

Connectivity. The overall wiring is summarised in Table 2, and the logical schematic is shown in Figure 4. Because the Enviro board is a HAT, it occupies the Raspberry Pi’s 40-pin header but re-exposes key interfaces on-board: 5V/3V3/GND rails, the primary I²C bus (SDA/SCL), and GPIO 4 (used here for the DHT22). The PMS5003 connects via the dedicated socket on the Enviro board, avoiding additional loose wiring.

Module	Signal	Pin	Notes
DHT22	Positive	5V	Supply (Enviro)
DHT22	Negative	GND	Common ground (Enviro)
DHT22	OUT	GPIO 4	Single-wire GPIO data (through Enviro)
SCD30	VDI	3V3	Supply (Enviro)
SCD30	GND	GND	Common ground (Enviro)
SCD30	TX/SCL	SCL	I ² C clock (Enviro)
SCD30	RX/SDA	SDA	I ² C data (Enviro)

Table 2. AirQ connectivity.

Prototyping. The first bench prototype with full connectivity is shown in Figure 5. A small port expander and a mini breadboard were used to provide quick terminations and probe points. These aids sped up bring-up and sensor swaps; they were omitted in subsequent configurations, which use direct connections when possible to reduce bulk and failure points.

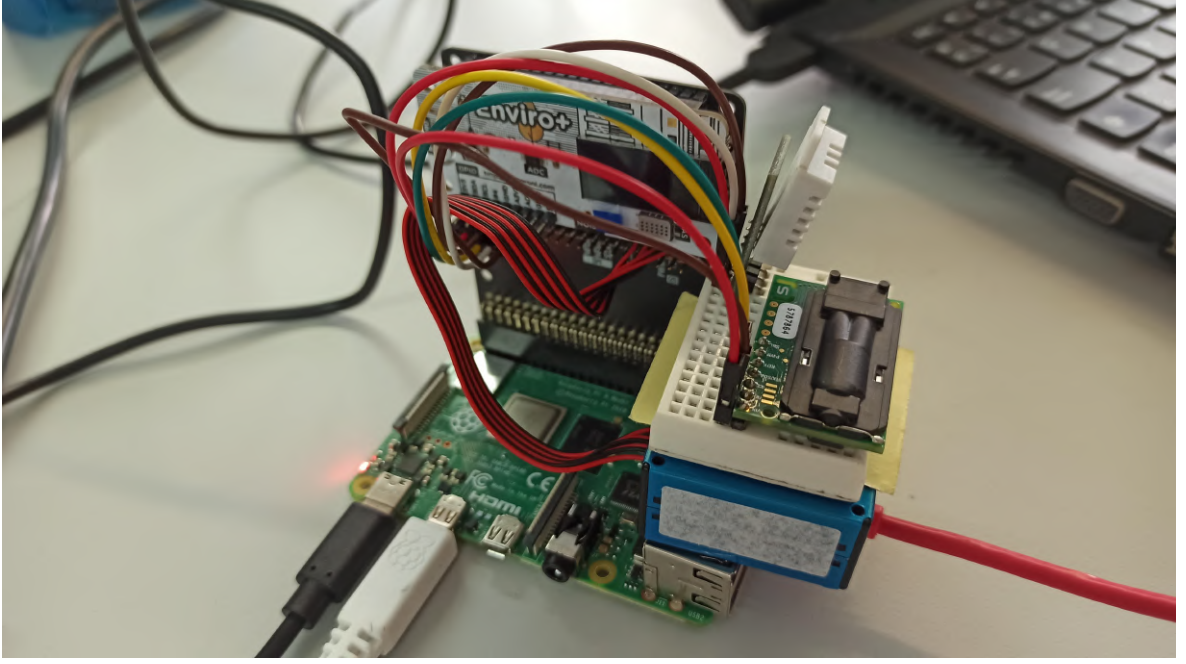
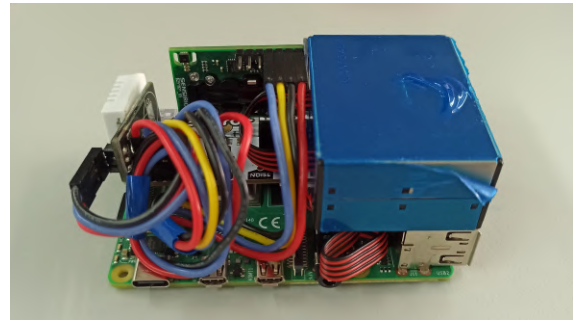


Figure 5. Early AirQ fully wired prototype.

A near-final bench build is shown in Figure 6. It differs only slightly from the definitive unit: the 3D-printed enclosure is absent and the jump wires are not yet fixed with tape to avoid accidental disconnections and short circuits. The components are folded to approximate the future chassis footprint and cable paths; the final prototype changes placements only minimally.



(a) Late prototype — view 1.



(b) Late prototype — view 2.

Figure 6. Two views of the late AirQ prototype.

Sensor mapping and redundancy. Header rows were soldered onto the Enviro HAT and the SCD30 to provide stable jump wire connections. Measurement redundancy was intentionally maintained to support accuracy gains in post-processing through weighted averages: temperature and humidity are captured by three distinct sensors. Table 3 summarises the sensors and their outputs. Several Enviro channels are out of scope and therefore disabled, notably the MICS6814 gas sensor and the on-board MEMS microphone.

Sensor	Measurements
BME280 [35]	Temperature, pressure, humidity
LTR-559 [36]	Illuminance
PMS5003	PM ₁ , PM _{2.5} , PM ₁₀ ($\mu\text{g}/\text{m}^3$); particle counts per 0.1 L: > 0.3, > 0.5, > 1.0, > 2.5, > 5, > 10
DHT22	Humidity, temperature
SCD30	CO ₂ , humidity, temperature

Table 3. AirQ measurements mapped to sensors.

Summary. Overall, the **hardware configuration** achieves a compact, reliable platform that reuses proven components from the ACTUA kit while focusing explicitly on air-quality monitoring. The deliberate simplification of subsystems and careful prototyping ensure a balance between robustness, maintainability, and clarity of purpose, providing a solid basis for the integration of the AirQ unit into the wider project.

3D Enclosures A custom two-piece PLA enclosure was designed and printed to house the reconfigured hardware. The **design aims** were compactness and robustness with minimal reliance on adhesives (applied in only one location to immobilise DHT22 sensor on its shelf), while preserving sufficient ventilation to avoid self-heating of the sensing elements. Sensor placement follows recommended orientations, with unobstructed exposure of the CO₂ and PM inlets and clear separation from heat sources. *Design-for-printability* was considered from the outset: geometries were planned to remain manufacturable as single parts where practical, avoiding forms that would require excessive supports or would be difficult to produce and operate.

Ease of assembly and service was also prioritised: components can be mounted and removed without difficulty, and although screws and standoffs are provided to tighten the enclosure, the structure is already sufficiently sturdy when assembled without them. The result is a compact, functional, and visually neat design that gives the kit a characteristic, well-finished appearance. Figure 7 shows the 3D models of the two enclosure parts.

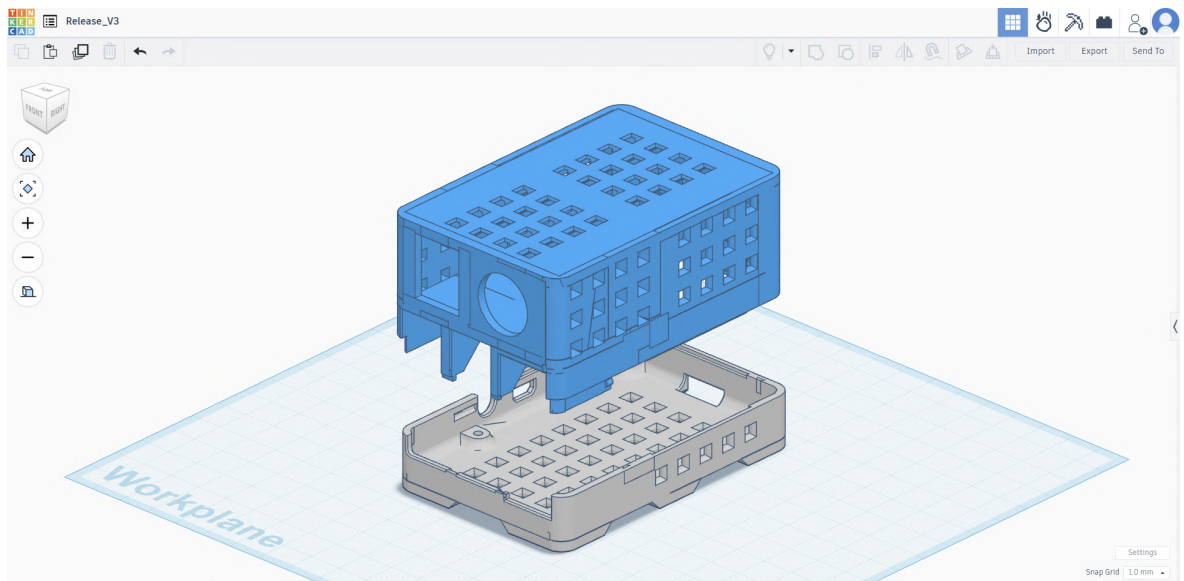


Figure 7. General view of the AirQ enclosure parts (top and bottom).

The **CAD workflow** was carried out in **Tinkercad** rather than more advanced tools such as Autodesk Fusion 360 or Inventor. Tinkercad was selected because it is free of licensing restrictions, entirely cloud-based, and therefore accessible across platforms. Its set of basic primitives and editing tools proved sufficient for the requirements of IoT prototype enclosures.

To **accelerate iteration**, the design process started from an existing Raspberry Pi chassis model [37] that was then heavily modified. Imperfections in the base model were corrected, and new structures were added to accommodate the specific sensors and components. Since Tinkercad lacks advanced sculpting features, modifications were implemented mainly through combinations of primitive shapes and Boolean operations (additions and subtractions).

A cornerstone of the design philosophy was adopting a **modular “LEGO-style” approach**. 3D models of the main components [38, 39, 40, 41, 42] were placed directly into the enclosure design, allowing an intuitive, visual method of aligning and shaping rather than relying on precise manual dimensioning of every detail. If an exact 3D model of a component was not available, a simplified representation was created from primitives, or the closest available model was adjusted to match the crucial dimensions. This approach provided flexibility and accelerated iteration, while also enabling early detection of a dimensional miscalculation in the base model, which was corrected before physical printing.

Figure 8 illustrates this process: the enclosure parts are shown in transparent view, revealing the embedded 3D models of the components — including sensors, the computing unit, and simplified jump wire placeholders that guided placement and routing.

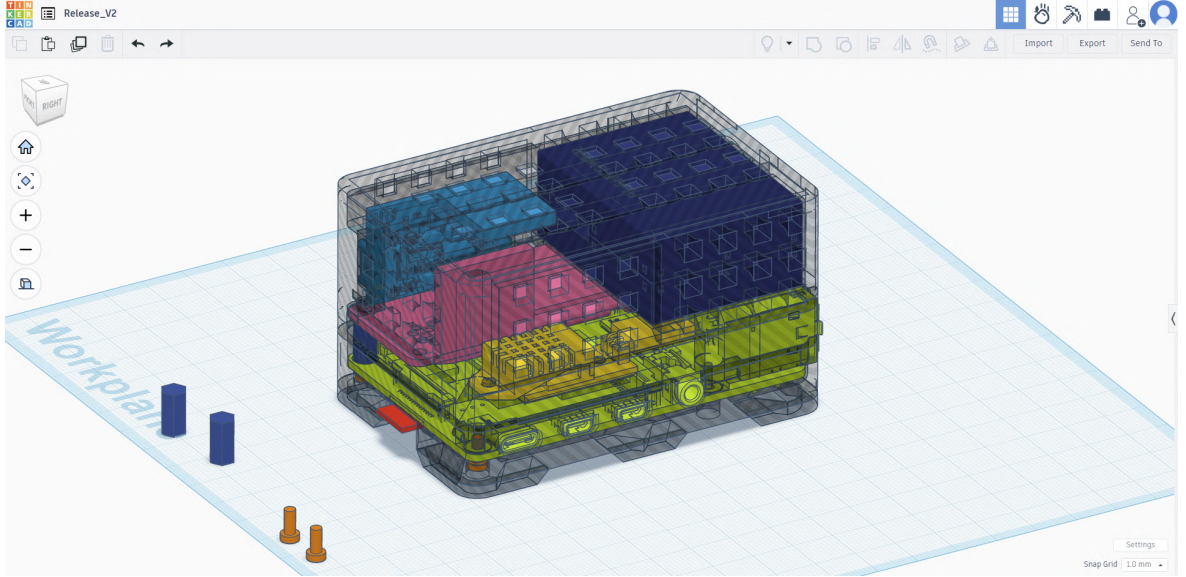


Figure 8. Transparent view of the AirQ enclosure showing 3D models of components.

Practical solutions were integrated into the compact layout to ensure reliable fixation of all sensors during assembly and easy removal for maintenance. For example, the SCD30 required a custom bayonet socket, as its geometry and clearance left no viable alternatives. Figure 9(a) shows the initial stage of this socket design, while Figure 9(b) presents the finished piece later integrated into the top enclosure.

The overall design also incorporates several practical features, illustrated in Figure 9(c): indentations to accommodate screw heads, puzzle-like joints to align the two

halves, a dedicated shelf for the DHT22, the previously mentioned bayonet socket for the SCD30, a raised bottom section for heat dissipation, and clips with corresponding holes to snap the two halves together.

Finally, the **ventilation pattern** pays homage to the metallic grid of the ACTUA project kit enclosure. Beyond its symbolic reference, this pattern reduces material use, lowers weight, and provides controlled flexibility to minimise cracking risk during repeated assembly and disassembly.

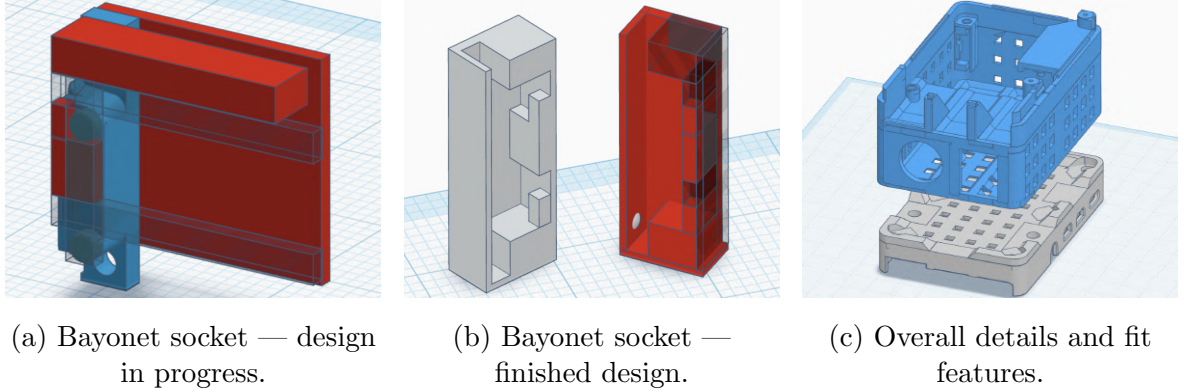


Figure 9. AirQ enclosure design: (a) bayonet socket concept, (b) finished socket design, and (c) multi-view highlighting interior details.

3D printing. After completing the CAD design, the enclosure models were exported in `.stl` format and prepared for fabrication using *Bambu Studio*, the slicing framework provided by Bambu Lab. In this stage, the two halves of the enclosure were arranged on the virtual printing plate, the printing material was selected, and supports were configured. A process of trial and error ensured supports were sturdy enough to fulfil their purpose while remaining easy to detach afterwards. Adhesion settings and infill density were also tuned for reliability, and part orientation was chosen to minimise support use while achieving sufficient stiffness.

For material, **PLA filament** was chosen, as it is a reliable option commonly used in comparable projects that provides consistent results. Printing was carried out on a **Bambu Lab P1S** printer, installed in the **URBots – Robotics Association of Universitat Rovira i Virgili**.

To cover the requirement of three complete AirQ kits, both halves of the enclosure were printed separately in batches of three. This approach enabled production of all necessary parts in only two final sessions and provided redundancy in case of printing failures (e.g., displacement or adhesion loss). For clarity and visibility, the two halves were intentionally printed in different colours; printing them in separate sessions also avoided mid-print filament changes, which would otherwise have increased material waste and printing time. As shown in Figure 10, (a) illustrates the top halves fully configured and ready for printing, while (b) shows the bottom halves prepared in the slicer. Table 4 summarises the main printing parameters, including total filament consumption and estimated printing time for each batch.

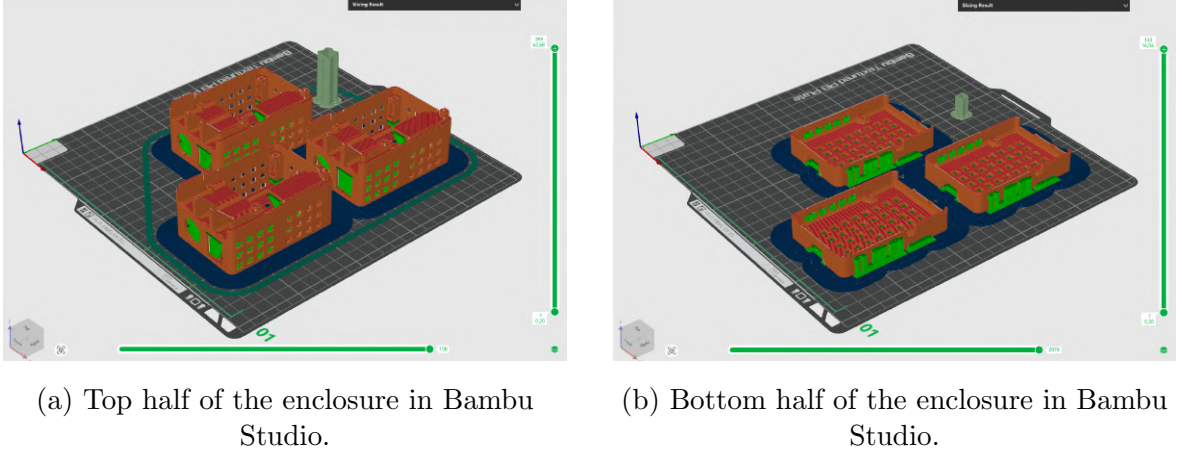


Figure 10. Preparation of the two enclosure halves in Bambu Studio before printing.

Enclosure part	Filament (g)	Printing time (h:mm)
Top half (3 units)	133	13:24
Bottom half (3 units)	66	5:32

Table 4. Summary of printing details for each half of the AirQ enclosure (batch of three units).

Summary. Overall, the **fabrication process** demonstrated that the enclosure design could be reliably reproduced with modest resources, producing sturdy and visually consistent parts suitable for field deployment.

Software The AirQ software runs on Raspberry Pi OS (formerly Raspbian) with a 32 GB microSD card providing operating system, code, and database storage. It is derived from the ACTUA kit firmware, which was fully reviewed and adapted to the present requirements. Non-essential features (remote control, network export) were removed, while support for the DHT22 sensor, calibration offsets, and simplified deployment scripts were added. The modular architecture of the original code was preserved, ensuring a lightweight and fault-tolerant system tailored to timestamped local logging.

System scripts. Deployment and supervision rely on a minimal set of shell scripts. `setup.sh` installs all dependencies, enables UART/I²C/SPI, and configures the system for reproducibility. At boot, `crontab` launches `start.sh`, which checks that all sensor processes (Enviro, PMS5003, SCD30, DHT22) are active and relaunches them if needed. Companion scripts (`stop.sh`, `check.sh`) simplify manual process management.

Configuration. Runtime parameters are defined in two INI files: `global.ini` (database, logging, sensor manager) and `sensors.ini` (individual sensor frequency, GPIO pin, port). These are parsed by `config_manager.py`, allowing reconfiguration without editing source code.

Database. All data are stored in an local SQLite database containing *id_sensor*, *id_parameter*, *value*, and *timestamp*. The database is created automatically if not present. Each record is time-stamped both as Unix epoch and in human-readable format. Storage demand is modest: with about 18 variables recorded every minute (≈ 1080 rows per hour), and each row occupying on the order of 60-80 bytes, the database grows at roughly 70 kB per hour (≈ 1.7 MB per day, ≈ 50 MB per month). This makes multi-week operation easily feasible on a 32 GB microSD card.

Logging. A rotating log system (`logger_manager.py`) records system activity and errors with time stamps, limiting file sizes and maintaining backups to avoid uncontrolled growth.

Sensor modules. Each sensor runs as an independent Python module (`enviro.py`, `PMS.py`, `SCD30.py`, `DHT22.py`), all following a shared template:

- Load configuration (frequency, tag, port, active state).
- Initialise the device using the appropriate driver library.
- Enter a loop: sleep for a random delay of 0–30 s, then record one measurement every minute.
- Write results to the database via `database_manager.py`.

In addition, each sensor process spawns a background thread with a UDP socket listener. This mechanism, inherited from the ACTUA baseline, was designed to allow runtime reconfiguration (e.g., changing sampling frequency) through inter-process messages. In the present Phase 1, no external commands are issued, so the listeners remain idle; however, they illustrate the modular architecture and preserve the option of co-ordinated control in later phases.

Sensor manager. The `sensor_manager.py` module acts as a watchdog: it verifies database health and ensures that each sensor process is alive and producing data. If a process stalls, it is terminated and restarted by `process_manager.py`, providing fault tolerance during long-term unattended operation.

Calibration and redundancy. Temperature and humidity are measured by three independent sensors (BME280, SCD30, DHT22). Offsets were derived through calibration against an external thermocouple and by comparing measurements with and without the enclosure. Corrections were applied in code, notably to mitigate self-heating of the Raspberry Pi and other electronic components. This redundancy improves robustness and allows cross-checks during later analysis.

Dependencies. The software relies on well-established Python packages chosen for simplicity and reliability. Sensor-specific libraries (`bme280` [43], `enviroplus` [33], `ltr559` [44], `pms5003` [45], `scd30_i2c` [46], `adafruit-circuitpython-dht` [47]) provide a clean abstraction layer, reducing the need for low-level code and ensuring stable operation. Hardware access is handled through `smbus`, `smbus2`, `psutil`, and `board`, while general-purpose libraries such as `numpy` and `pandas` support efficient data handling and analysis.

Summary. Overall, the installed and configured software boots automatically and relies on the Raspberry Pi OS for periodic Wi-Fi time synchronisation. Each sensor executes its own acquisition loop with one-minute periodicity and randomised startup delay, writing calibrated measurements into a local SQLite database under continuous supervision. The result is a compact, robust platform focused exclusively on reliable, audit-ready IEQ data logging.

3.3 Noise

Hardware The Noise kit was designed from scratch as a dedicated platform for acoustic analysis. In contrast to the AirQ unit, almost no legacy hardware was reused; instead, carefully selected components were integrated to achieve high-frequency audio

acquisition, efficient processing, and compact form factor suitable for indoor deployment.

Component selection. All core components are listed in Table 5. Most modules are Adafruit variants with well-documented pinouts, datasheets, and usage examples available from the manufacturer’s website and GitHub repositories. This choice simplified integration, ensured hardware–software compatibility, and accelerated prototyping. The ESP32-S3 Feather was selected as controller for its dual-core architecture, integrated Wi-Fi/BLE, built-in LiPo charging circuit and connector, and 2 MB of PSRAM, which later proved advantageous for buffering and processing data. A known drawback is the relatively noisy ADC subsystem, which Espressif acknowledges and mitigates through techniques such as multisampling and the use of a 100 nF bypass capacitor on the input [14]. The MAX9814 amplifier combined with an electret microphone was recycled from the ACTUA project kit and chosen for its suitable frequency response to cover the major human voice range ($\approx 70\text{--}10,000\text{ Hz}$). A compact TFT display with an embedded microSD slot was added to provide visual feedback while simultaneously offering local storage, combining two key functions in a single module. A 3500 mAh LiPo battery was selected to provide sufficient autonomy during contextual tests, and a simple pushbutton was included to enable direct user interaction.

Component	Role	Source	Notes
Adafruit ESP32-S3 Feather (4MB Flash, 2MB PSRAM) [48, 49]	Controller, processing unit	New	Dual-core; integrated BLE/Wi-Fi; LiPo charging and connector; USB-C
MAX9814 amplifier with electret microphone [50, 51]	Audio input	Recycled	Analog output to ADC; gain fixed at 40 dB (pin tied to 3V)
Adafruit 0.96" 160×80 TFT + microSD slot (ST7735) [52]	Display + Storage	New	SPI interface; microSD slot for logging
3500 mAh LiPo battery	Power	New	Autonomy
Mini pushbutton	User input	New	Used with internal pulldown resistor
microSD card	Storage	New	Audio derived data logging

Table 5. Noise kit core components.

Connectivity. Pin assignments and wiring are summarised in Table 6, with the complete schematic shown in Figure 11. The microphone output is routed to GPIO 8, which maps to an ADC channel not shared with Wi-Fi [14], reducing conflicts during concurrent sampling and connectivity. The TFT display and microSD slot are accessed over a shared SPI bus, with separate chip select lines. The pushbutton connects between 3.3 V and GPIO 6, using the ESP32’s internal pulldown resistor to avoid spurious triggers.

Module	Signal	Pin	Notes
Microphone (MAX9814)	OUT	GPIO 8	ADC input
TFT (ST7735)	CS / DC / RST / LIT	GPIO 9/11/10/13	SPI control + backlight
TFT + microSD	SCK / MISO / MOSI	GPIO 36/37/35	Shared SPI bus
microSD	CS	GPIO 12	Separate chip select
Pushbutton	OUT	GPIO 6	Internal pulldown resistor enabled
Power	LiPo / USB-C	Feather PCB	Onboard charging with limited current [53, 54]

Table 6. Noise kit connectivity.

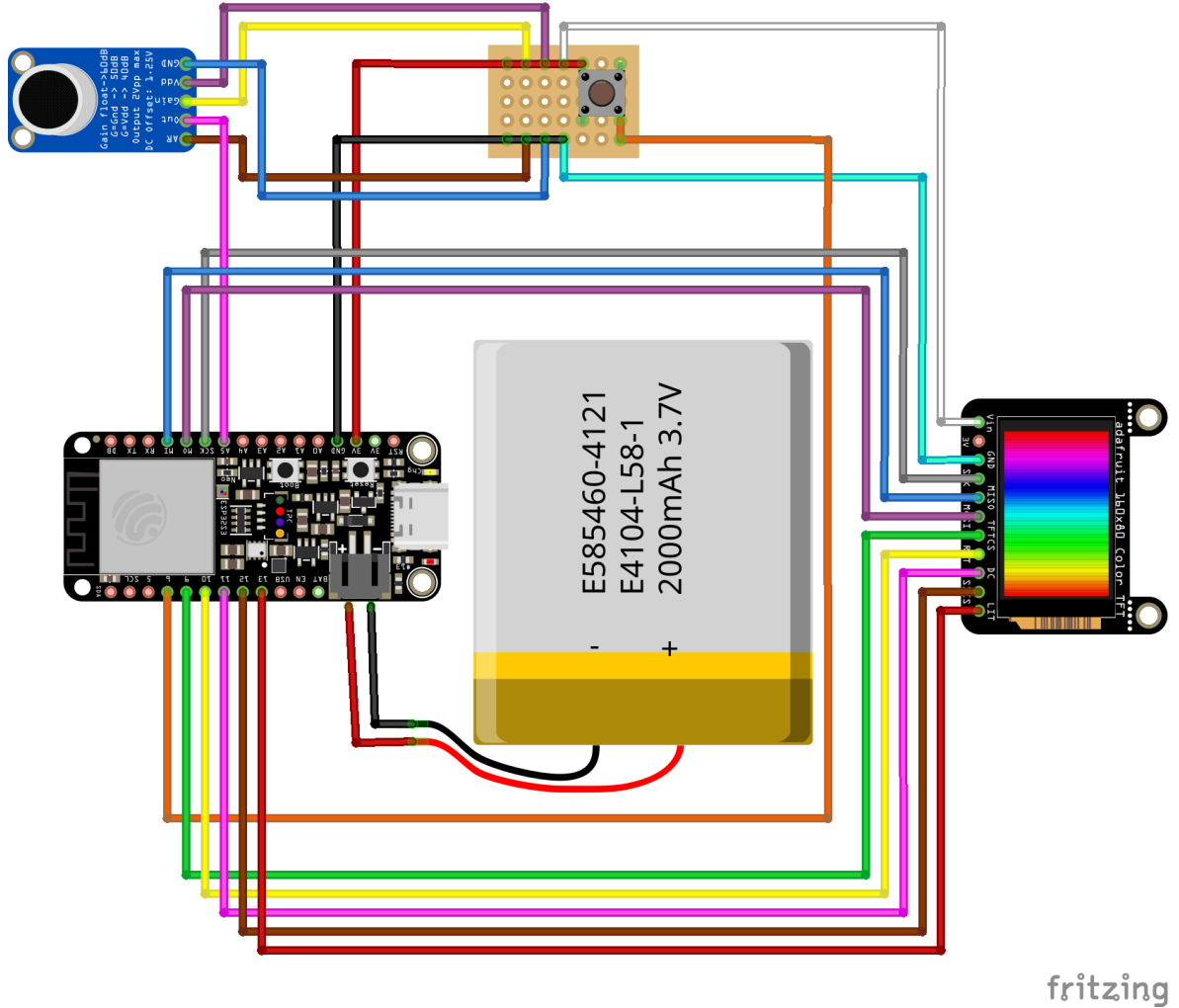


Figure 11. Noise kit logical schematic and wiring [55].

Prototyping. Two stages of prototyping were carried out, shown in Figure 12. The early fully wired prototype (a) used a breadboard to provide quick terminations and probe points, which simplified bring-up and sensor swaps at the cost of some bulk, similar to the AirQ kit. These aids were later omitted in favour of direct wiring to reduce failure points. The late-stage prototype (b) reproduced the final layout without

the 3D-printed enclosure; at this stage, jump wires were bundled together to achieve a cleaner implementation, but individual connections were not yet taped down. This arrangement facilitated testing while avoiding premature fixation, allowing validation of ADC performance, SPI speed, and button debounce before integration into the final chassis.

Additional assembly work included soldering header rows onto the ESP32 and the TFT breakout PCB to ensure reliable jump-wire connections. A small 5×7 perfboard was also used to mount the pushbutton and redistribute power and reference lines (3 V and ground). To compactly integrate the controller with the battery, reclosable fastener tape was applied between the underside of the ESP32 PCB and the top of the LiPo cell, keeping both units securely attached while still allowing them to be separated if required. Final taping of connections to prevent accidental disconnections and short circuits was reserved for the enclosure stage.

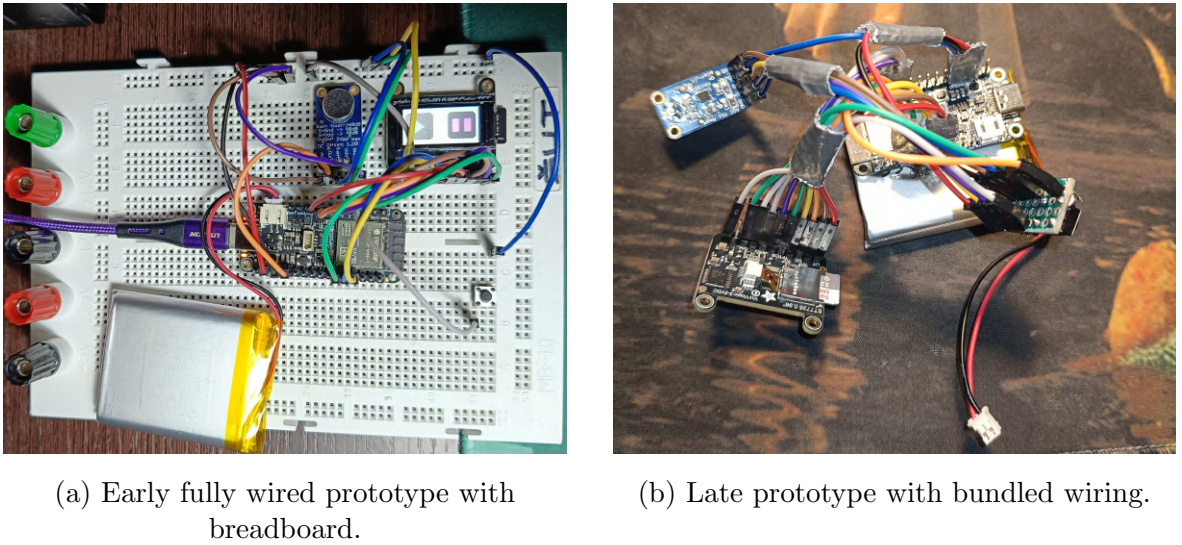


Figure 12. Noise kit prototyping stages: (a) early prototype using a breadboard for bring-up and (b) late-stage prototype without enclosure.

Additional considerations. The ESP32-S3 provides BLE and Wi-Fi connectivity, enabling future expansion to distributed sensing and coordination. Its dual-core design supports true concurrency under RTOS, which is critical for overlapping tasks such as sampling, FFT processing, and wireless communication. The SoC also supports capacitive touch sensing on some unused GPIO pins, which may be exploited in future iterations to complement the pushbutton. Although the kit can operate on battery power, its functionality is computationally and energetically demanding; therefore, future deployments are expected to favour connection to a constant power source by cable, with the battery retained primarily for testing and short-term autonomy.

Summary. Overall, the **hardware configuration** of the Noise kit provides a compact, energy-efficient, and extensible platform for acoustic monitoring. The deliberate selection of components and careful prototyping ensure reliable audio capture, real-time processing capability, and clear upgrade paths toward coordinated multi-unit deployments in Phase 2.

3D Enclosures A custom two-piece PLA enclosure was designed and printed to house the Noise kit hardware. In contrast to the AirQ case, which evolved from a

modified base model, this enclosure was created **entirely from scratch** in **Tinkercad** using basic primitives, previously elaborated details and Boolean operations. Despite the simpler toolset, the workflow met all requirements and yielded a robust, compact chassis.

The enclosure adopts a **rectangular, compact form factor** and consists of two interlocking halves. No screws are used: the halves are held together by **clips and mating recesses** that produce a firm lock. Most components are constrained directly by the internal geometry; the LiPo battery is the only element fixed with *double-sided gel adhesive* to the bottom half. The design balances tight packaging with serviceability, ensuring secure fixation in operation and straightforward disassembly when maintenance is required. Figure 13 shows the 3D models of the two enclosure parts.

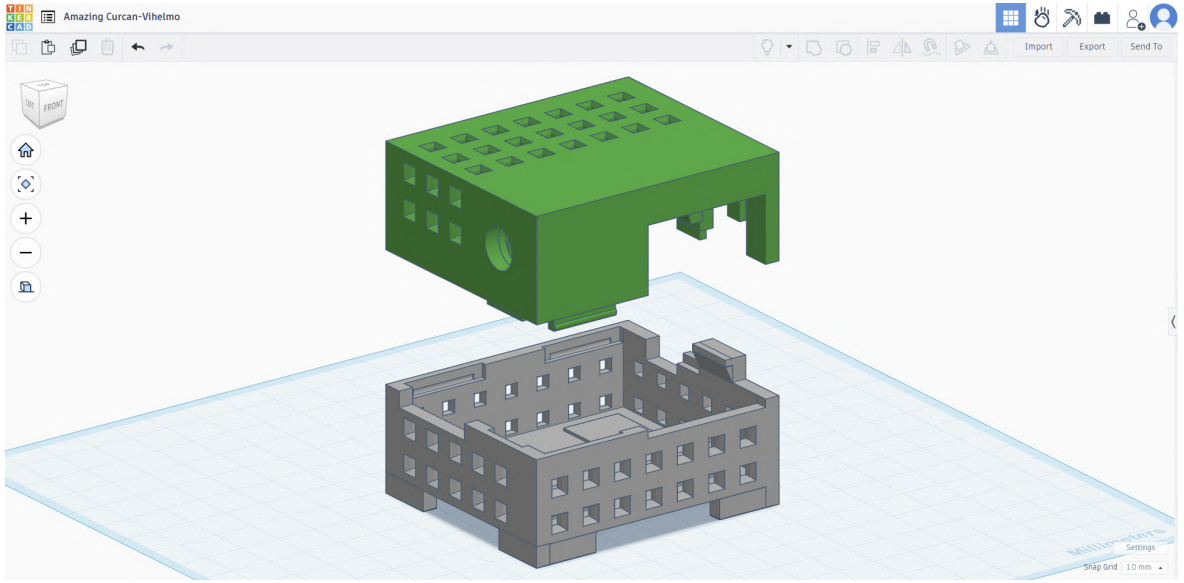


Figure 13. General view of the Noise enclosure parts (top and bottom).

CAD workflow. The CAD process remained intentionally lightweight. As in the AirQ case, a **“LEGO-style” placement** was preferred: exact or simplified 3D representations of components [56, 57, 58, 59] were dropped into the scene and the enclosure was shaped around them, minimising manual measurement and reliance on datasheet drawings. Where this was not feasible—e.g., for **custom elements** (snap features, rails, stoppers) or for devices with **no reliable 3D model**—key dimensions were taken from the physical unit or the datasheet and recreated as simple volumes to preserve clearances. This approach supported rapid iteration and early detection of interferences before printing. A transparent overlay with embedded component models is shown in Figure 14.

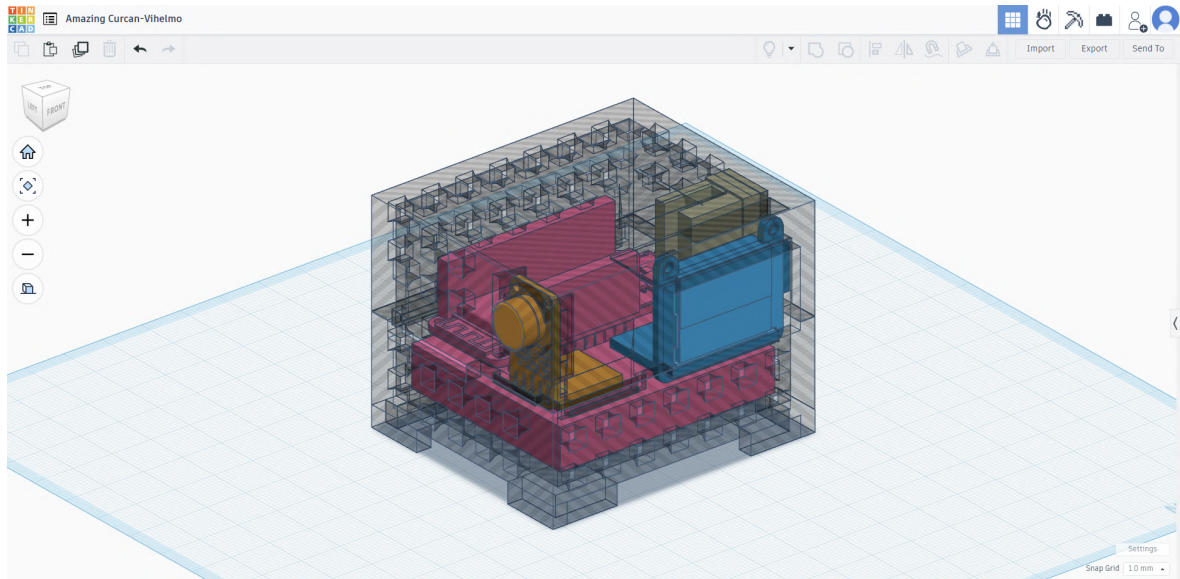


Figure 14. Transparent view of the Noise enclosure showing component placement and clearances.

Key features. The top half integrates an aperture for the microphone unit that **snaps in** and is retained by **moulded** bumps. A lateral cut-out provides visibility for the TFT screen, which is held by inserting its **mounting protrusions** into matching internal slots of the top half. The rear face includes a recessed opening for **microSD access** without separating the halves. The pushbutton is accessible through a small aperture and is **flush with the surface** to reduce accidental presses. To locate the button firmly yet serviceably, the perfboard slides along a **rail** in the top half and locks against a **stopper shelf** in the bottom half once assembled. Ventilation openings are placed on the top and sides; the bottom surface remains **solid** to reduce puncture risk for the battery. Finally, a small **antenna window** aligned with the ESP32-S3 antenna minimises signal attenuation—particularly if future filaments include conductive pigments—ensuring reliable BLE/Wi-Fi even under low transmit-power settings. Figure 15 summarises these elements: (a) interior details of the enclosure half, and (b) the 3D model of the perfboard block with soldered pushbutton and imitation of rigid jump wire connectors, illustrating how the assembly slides into the rail and locks in place.

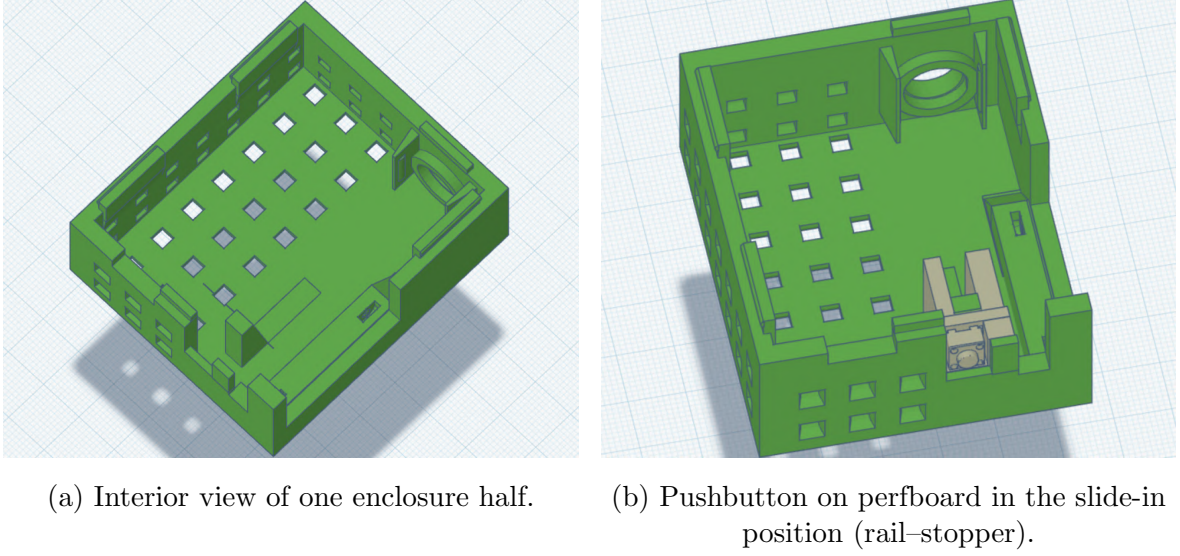


Figure 15. Noise enclosure details: (a) interior features; (b) pushbutton perfboard assembly and rail-stopper sliding mechanism.

3D printing. Models were exported in `.stl` and prepared in *Bambu Studio*. Prior AirQ slicer settings were reused with minor adhesion adjustments. **PLA filament** was selected. The initial spools were discarded due to suspected humidity; a different spool was used for final parts, leading to a colour difference across units. Printing was performed on a **Bambu Lab P1S** at the URBots workshop. Three complete enclosures were produced by printing **batches of three** for each half within the same session. Figure 16 shows the slicer layouts, and Table 7 summarises material use and time per batch.

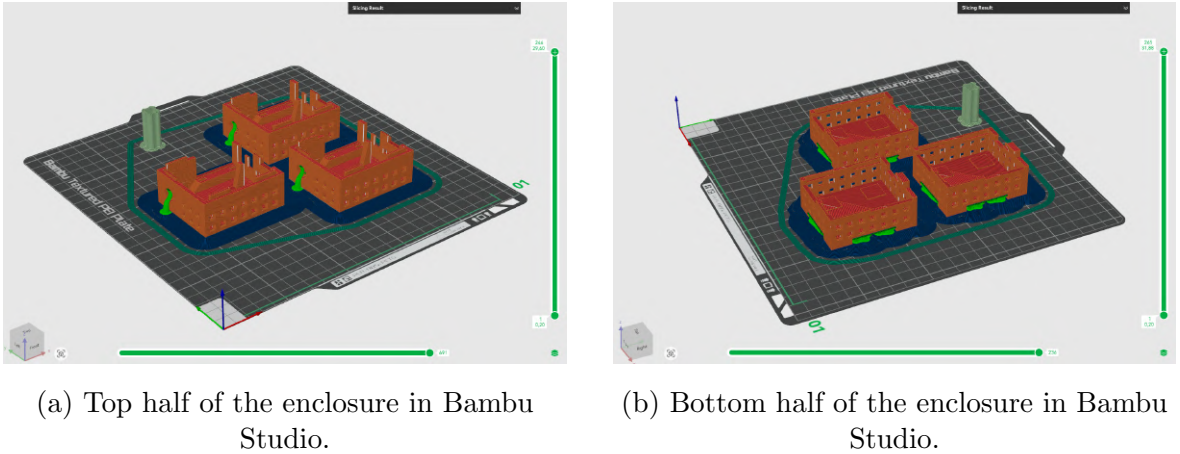


Figure 16. Preparation of the two enclosure halves in Bambu Studio before printing.

Enclosure part	Filament (g)	Printing time (h:mm)
Top half (3 units)	72	5:56
Bottom half (3 units)	76	7:15

Table 7. Summary of printing details for each half of the Noise enclosure (batch of three units).

Summary. Overall, the **fabrication process** confirmed that the Noise enclosure

is reproducible with modest resources, yielding sturdy, compact, and visually consistent parts tailored to the acoustic monitoring use case.

Firmware The Noise kit firmware was developed from scratch in the Arduino IDE using the Arduino ESP32 core [60]. This toolchain provides a fast edit–build–flash loop while exposing ESP-IDF facilities (FreeRTOS, continuous ADC with DMA, Wi-Fi/BLE, PSRAM). After validating each peripheral in isolation (microphone/ADC path, TFT, microSD, Wi-Fi/BLE radio, pushbutton, fuel gauge), the codebase converged to a modular, deterministic design. Run-time parameters (sampling, FFT, pin map settings) live in `signal_config.h`; per-module knobs (debouncing, feature thresholds, debug flags) live with each module. VS Code/PlatformIO and MicroPython/CircuitPython were evaluated but not adopted due to added complexity or tighter latency/memory/low-level constraints.

Chronology and design choices. Early prototypes used a tight loop (read ADC $\rightarrow$ delay) that blocked concurrency, limited sample rate, and raised CPU/power. microSD trials showed clear gains from large writes and avoiding frequent open/close. The final architecture adopts continuous-mode ADC with DMA, A/B double buffering in PSRAM, explicit FreeRTOS task orchestration, and buffered, sector-aligned logging. Signal analysis uses overlapping transforms across each audio frame (a merged short-time Fourier transform, STFT), matching timing and memory budgets without blocking other tasks.

Software flow. Each cycle does three things: *capture* a clean slice of sound, *analyse* it, and *store* what matters. Work that is not time-critical waits for the gap between cycles so the acquisition path stays clean. At start-up the device briefly enables Wi-Fi to synchronise time and then keeps that clock; if time is not yet confirmed, logging is deferred to avoid wrong timestamps. Sampling has priority and runs to completion; analysis and logging occur between captures and never delay the next one. UI, BLE, and housekeeping also use this idle window. Modules exchange small, bounded messages; there are no long blocking calls on the real-time path. *Errors are contained at source*: each module retries with a bounded budget and backoff; if recovery fails, the orchestrator reinitialises the module and, only as a last resort, requests a controlled restart to keep the device autonomous.

Runtime architecture. A FreeRTOS software timer enforces a fixed cycle of $T_c \approx 1.5$ s with four cooperative tasks:

1. *Sampler* — acquires a 500 ms window at 44.1 kHz (22,050 samples; measurable content up to ≈ 22.1 kHz) into PSRAM and hands off a ready buffer.
2. *FFT* — performs overlapping 4096-point analyses (step 2048), derives voice-band features, and estimates the dominant bin.
3. *Logger* — appends compact binary frames to microSD using 512 B-aligned buffered writes, rolling files, and safe recovery.
4. *Battery/UI* — owns the pushbutton, battery/TFT view, backlight control, and on-demand BLE publish of the *last frame*’s dominant peak.

Sampler $\rightarrow$ FFT $\rightarrow$ Logger are coordinated by direct task notifications; blocking I/O stays off the acquisition path. ADC1 is used to avoid the radio contention associated with ADC2. Time synchronisation is done at boot and periodically; until the clock is

confirmed valid, time-dependent actions are gated. All tasks run on one core by design; idle-friendly delays allow light sleep in inactive periods. Watchdog-safety is ensured by bounded waits and periodic yields; heavy work is kept out of the acquisition path.

Acquisition. Audio capture uses the ESP32 continuous ADC with DMA [61] and two PSRAM windows (A/B), implemented in `audio_sampler.h/.cpp`. The default path uses bounded polling to drain the DMA ring with short timeouts (watchdog-safe) and periodic yields. When the target count is reached, acquisition stops, A/B buffers swap, the window is marked ready, and FFT is triggered. An alternative interrupt-driven path (kept as commented reference) uses a conversion-complete callback so the sampler task collects the entire frame as a unit. At initialisation the sampler enables curve-fit calibration (raw values→mV) and can optionally enable the on-chip IIR filter (coefficient 4) behind a build flag. A small WAV export helper exists for calibrated spot checks during development. If the ADC or DMA reports an error, the sampler is rearmed; persistent failure triggers a local reinitialisation before any broader restart is considered.

Signal processing. For each frame the engine performs DC removal, applies a Hamming window, computes magnitudes, and merges overlapping slice outputs (variation of STFT): in 100–4000 Hz it retains per-bin maxima with a small gate; outside that band it averages across slices. From the combined spectrum it computes RMS in-band/out-of-band, a linear SNR, spectral flatness, contrast, peak count, a dominant-frequency estimate, and an intensity score based on rise over an adaptive baseline. It also sets a *binary voice-present flag* using a short confirmation step and hysteresis so brief spikes do not toggle state. All bands, gates, and thresholds are compile-time configurable in `signal_config.h`. Storing the combined spectrum per frame enables offline re-processing with alternative detectors without firmware changes.

PSRAM usage. A/B sample windows and all FFT working arrays live in PSRAM. This preserves internal SRAM for stacks, drivers, and ISRs, and avoids fragmentation from large transient buffers.

Storage and logging. Each processed frame is written as a compact binary record with tag ("FFT2"), Unix timestamp, voice flags/metrics, bin count, and interleaved {frequency, magnitude} pairs. Frames are padded to 512 B boundaries and appended via a persistent buffer; the log file remains open, and indices/state are updated atomically in batches to reduce wear. At boot the logger discovers the highest `/LOG_####.BIN` and reconciles offsets so appends are always safe. Writes are blocked until time is both synced and sane (guard floor) to avoid mis-dated data; last good epoch is persisted in NVS. *Power-loss safety:* writes are grouped so an abrupt outage may drop at most the last buffered frame, while previously committed data and index/state remain consistent. On I/O errors the logger retries, reopens, and reinitialises the card; if the card is absent it degrades to “no logging” while sampling and BLE continue.

User interaction & power. GPIO 7 controls the STEMMA I²C rail used by both the MAX17048 battery monitor and external peripherals [49]; it is enabled only during measurement or UI activity. The pushbutton employs the internal pulldown and an ISR latch, with debounced requests served between acquisition cycles. On button press the Battery/UI task updates the TFT with battery status and, if a recent FFT is available, *triggers BLE*: a short on-demand advertising window (≈ 10 s) is opened, and upon connection the *last frame's* dominant frequency and magnitude are published. The link remains active until the client disconnects. Low-battery thresholds ensure

that under critical conditions the system issues a warning and enters deep sleep to protect the Li-Po.

Development-only tooling. During prototyping, a lightweight WebSocket server with an HTML page streamed live frames and the sampler’s short WAV snippets for real-time listening, which accelerated gain/threshold tuning and QA. In addition, a Python script reconstructed frames from the microSD binary log for offline verification. Figure 17 shows a frame plotted from a microSD binary log using the Python script, validating the log format. Figure 18 shows a frame received with the WebSocket tool during tuning. This path is disabled in production but retained as stubs. *Observation.* Curiously, WebSocket audio sounded clean with the microSD inserted but noticeably distorted without it—likely due to clock/sleep jitter or floating SD lines.

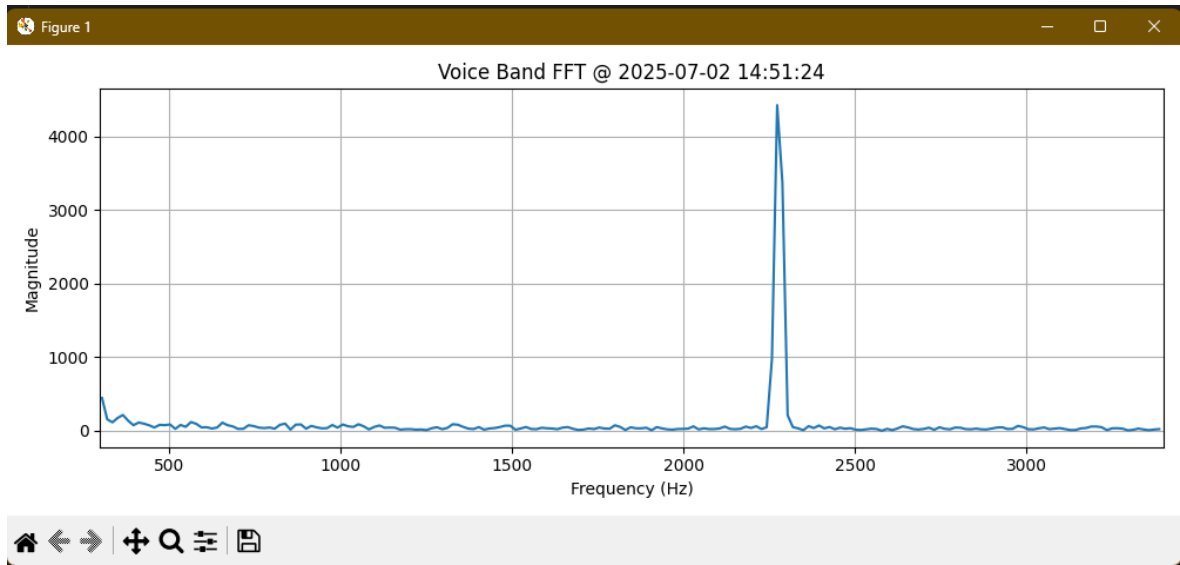


Figure 17. Frame plotted from a microSD binary log.

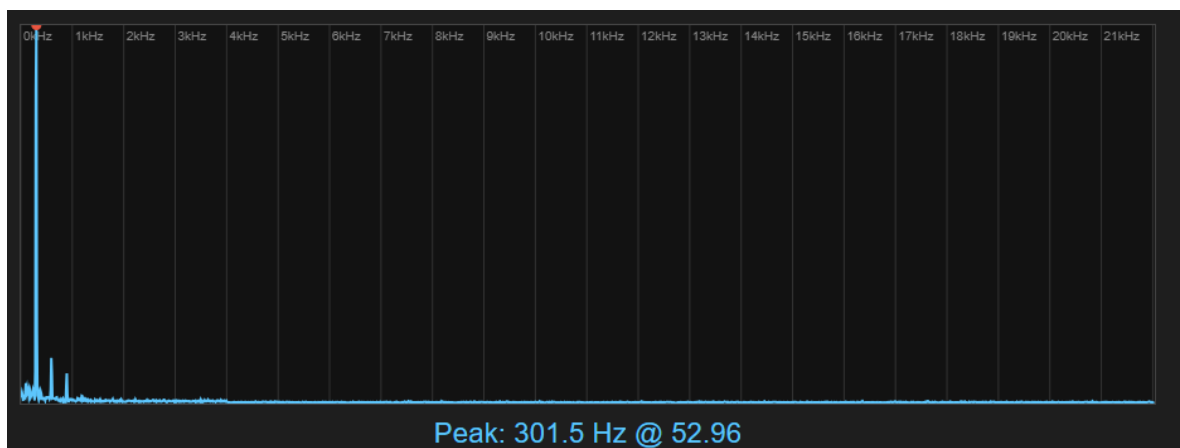


Figure 18. Frame received via the WebSocket tool.

Module layout & public API.

- `audio_sampler.h/.cpp`: continuous ADC + DMA; bounded polling (default); optional interrupt path; PSRAM A/B; calibration; optional IIR; WAV helper.

- `fft_engine.h/.cpp`: overlapping 4096-pt FFT with Hamming; combined spectrum (merged STFT); voice-band features (100–4000 Hz); dominant bin; intensity over adaptive baseline.
- `fft_logger.h/.cpp`: 512 B-aligned buffered frames; time gating; rolling files; discovery/recovery; atomic index/state.
- `wifi_manager.h/.cpp`: Wi-Fi connect/recover; NTP sync; last-epoch persistence (NVS); time sanity helpers; bounded backoff.
- `button_handler.h/.cpp`: ISR hook, debounced polling; idle-window UI requests.
- `battery_monitor.h/.cpp`, `display_manager.h/.cpp`: MAX17048 battery status reads with STEMMA power-gating; battery UI; backlight; *triggers BLE peak publish on demand*.
- `ble_fft.h/.cpp`: short on-demand advertising; dominant peak reporting (invoked by Battery/UI).
- `signal_config.h`: central run-time parameters (sampling, FFT, pins).
- `Mic_Modular_Code.ino`: orchestrator (timer $\rightarrow$ sampler $\rightarrow$ FFT $\rightarrow$ logger); notifications, time sync, UI flow and recovery decisions.

Dependencies. The firmware relies on well-established Arduino and ESP-IDF libraries selected for stability and hardware support. For digital signal processing, `arduinoFFT` [62] provides efficient frequency-domain analysis. Display functionality is supported by `Adafruit_GFX` [63] together with `Adafruit_ST7735_and_ST7789_Library` [64]. Battery monitoring uses `Adafruit_MAX1704X` [65]. Storage access is handled through SD [66], FS, and SdFat [67]. Core microcontroller I/O is managed with SPI and Wire, while connectivity and time synchronisation rely on WiFi, BLE, and ESP-IDF SNTP bindings.

Storage budget. With $N=2048$ bins and single-precision pairs, a raw frame is $32 + 2 \times N \times 4 \approx 16,416$ B; 512 B alignment yields $\approx 16,896$ B (≈ 16.5 kB). At a 1.5 s cycle: ≈ 0.68 MB/min, ≈ 38.7 MiB/h, ≈ 0.91 GiB/day.

Summary. The firmware follows a streamlined sampling $\rightarrow$ processing $\rightarrow$ logging pipeline that sustains the 1.5 s cycle with margin. Efficiency is achieved through single-core scheduling, non-blocking design, and selective peripheral use. A pooled and merged STFT yields consistent per-frame spectra, enabling exploration of system capabilities and configuration for high-quality data suitable for later analysis. *Robustness* stems from error containment, bounded retries, and power-loss-safe logging, so the device continues to capture and interpret signals even under faults.

3.4 Data Preparation and Analysis Software

All preparation and analysis code is written in Python. Workflows run from *VS Code* (with *Geany* for quick edits). *DB Browser (SQLite)* is used only to inspect tables and, when convenient, export/import CSVs. A single YAML file (`time_window-unified.yaml`) fixes the study timezone and window so every stage uses identical bounds; an optional `weights.yaml` provides per-sensor weights for temperature and humidity fusion. Intermediate artefacts are saved as columnar `.parquet` files.

Chronology and design choices. Early prototypes exported *Noise* spectra into CSV or .db files, which were slow to process and yielded cluttered, unreliable outputs due to flawed initial algorithms. The current pipeline reads *Noise* spectra *directly from binary logs* and computes only the features needed for minute-level speech proxies. For *AirQ*, each unit’s records are pulled from SQLite over the unified window, temperature and humidity are fused with weights, and the results are written to a consolidated minute table. Time handling is explicit end-to-end: timestamps are localized at ingestion and all subsequent computation runs on a unified minute grid. Re-running any stage simply refreshes its outputs.

Software flow. The pipeline converts raw files into two clean, minute-level tables and asks a simple question: *do short changes in speech activity precede shifts in air metrics?* Local time is standardized to UTC, and for each kit a complete minute grid is built so gaps remain explicit.

AirQ data is prepared by fusing redundant temperature and humidity channels *row-wise* using `weights.yaml`, producing a single TEMP and HUM value per record. All variables are then aggregated per kit and per minute. The fusion step also records, for bookkeeping, the number of contributing sensors and the total samples used in that minute.

Noise spectra are decoded directly from the device’s binary `LOG_####.BIN` files. For each frame the code derives band energy (100–4000 Hz), out-of-band energy, linear SNR, spectral flatness, and a short rise over an adaptive baseline. A brief multi-frame confirmation gate yields a binary speech flag. Minute roll-ups include coverage (`frames`, `coverage_s`, `coverage_rate`), the primary speech proxy (`voice_rate_time` – fraction of acquired seconds flagged as speech), and stability summaries (`intensity_mean`, `snr_mean`, `sfm_mean`, `bandRMS_mean`, `noiseRMS_mean`).

The two minute tables are then aligned by kit and UTC minute and passed through a light coverage gate.

Scopes and testing. Three scopes are analysed: **ALL** (entire window); **OCCUPIED** (CO₂ proxy: above a low baseline or gently rising); and **PRESENCE-CTRL** (within OCCUPIED, slow presence and daily rhythms are removed from *both* streams by regressing on a low-pass CO₂ baseline and hour-of-day sine/cosine). For each air endpoint and scope, short *forward* lags (Noise leads Air) are scanned, and Pearson correlations are computed with autocorrelation-aware *p* values via an effective sample size. Benjamini–Hochberg FDR is applied across the tested forward lags (and across multiple noise variables if those optional metrics are enabled; by default only the primary proxy is used). Reverse (negative) lags are written for diagnostics but excluded from the FDR family. Fisher-*z* CIs accompany headline *r* values.

Stage architecture.

1. *Ingestion & normalization.* Local timestamps (Europe/Madrid) are localized and converted to UTC at load. For each kit, a complete, gap-aware minute grid spanning the YAML window is built; measurements are attached to this grid with no silent filling.
2. *AirQ preparation (minute level).* Retained variables: CO₂, PM₂₅, PM₁₀, TEMP, HUM, PRES, LLUM. Processing: (i) *row-level* fusion of redundant TEMP/HUM channels

using `weights.yaml` to produce a single `TEMP` and `HUM` per record; (ii) per-kit, per-minute aggregation (means and sample counts) for all retained parameters; (iii) merge per-kit tables into one dataset aligned to the minute grid. *Output:* `data/processed/airq_1min.parquet`. (The smoothed CO₂ change `dCO2_mean3` is derived later during analysis.)

3. *Noise preparation (frame $\rightarrow$ minute).* Parse binary spectra; for each frame compute band/out-of-band energies, linear SNR, spectral flatness, and baseline rise. A short confirmation gate emits a speech flag. Minute roll-ups: coverage (`frames`, `coverage_s`, `coverage_rate`), speech proxies (`voice_rate`, `voice_seconds`, `voice_rate_time`), and stability summaries (`intensity_mean`, `snr_mean`, `sfm_mean`, `bandRMS_mean`, `noiseRMS_mean`). *Output:* `data/processed/noise_voice_1min.parquet`.
4. *Alignment & scopes.* Inner-join AirQ and Noise minute tables by kit and minute; apply light quality gates. Produce **ALL**, **OCCUPIED** (CO₂ proxy) and **PRES-ENCE-CTRL** (residualized) scopes.
5. *Lag-correlation engine.* For each Air endpoint and scope, compute Pearson’s r over forward lags (levels: 0–10 min; derived like `dCO2_mean3`: 0–20 min). Use 1% per-tail trimming, autocorrelation-adjusted inference (effective N), BH-FDR across the forward-lag family, and Fisher- z CIs. Emit per-lag CSVs (forward and reverse), a headline row per endpoint/scope (best forward lag by minimum q_{BH} , then adjusted p), a concise `summary.md`, and `diagnostics.json`.

Variables considered. *AirQ:* CO₂ (and its short-horizon change `dCO2_mean3`), PM₂₅, PM₁₀, `TEMP`, `HUM`, `PRES`, `LLUM`. CO₂ is the primary room-scale tracer for human presence and ventilation: levels reflect accumulated exhaled CO₂, while the smoothed derivative (`dCO2_mean3`) is more sensitive to short-lag changes, which is central to our timing hypotheses. PM_{2.5}/PM₁₀ capture activity-related resuspension and general aerosol load; they are analysed as secondary endpoints given their episodic nature and device/environment dependence. `TEMP`/`HUM` are included to characterise the thermal-moisture background and to aid interpretation; they are not headline outcomes. Particle-number channels and PM₁ are retained for diagnostics. *Noise:* the **primary** proxy is `voice_rate_time` (share of the actual minute with voice present). Optional robustness metrics (e.g., `intensity_mean`, `snr_mean`) exist but are disabled by default, so headline tests rely on `voice_rate_time`.

Outputs.

- *Prepared datasets* (exactly one row per kit per minute over the configured window):
 - `airq_1min.parquet`: {CO₂, PM₂₅, PM₁₀, `TEMP`, `HUM`, `PRES`, `LLUM`} plus fusion bookkeeping fields `TEMP_sources`, `HUM_sources`, `TEMP_samples`, `HUM_samples`, and timestamps `ts_min_utc` (and `ts_min_local` if emitted).
 - `noise_voice_1min.parquet`: coverage (`frames`, `coverage_s`, `coverage_rate`), speech proxies (`voice_rate`, `voice_seconds`, `voice_rate_time`), stability metrics (`snr_mean`, `sfm_mean`, `intensity_mean`, `bandRMS_mean`, `noiseRMS_mean`), with `ts_min_utc`.

- *Per-lag scans* (`reports_clean/<AIR_KIT>_<NOISE_KIT>/`): CSVs per endpoint and scope with lag (min), Pearson r , N_{eff} , 95% CI (Fisher z), one- or two-sided adjusted p , BH-FDR q , r^2 , and Spearman ρ (cross-check). Forward and reverse (diagnostic) scans are written separately.
- *Headline summary* (`headline_results.csv`): one row per endpoint/scope with the best forward lag (by minimum q_{BH}), its r , CI, adjusted p/q , and the decision.
- *Narrative report* (`summary.md`): concise overview of window, scopes, endpoints, significant forward lags, and key values.
- *Diagnostics* (`diagnostics.json`): window bounds, minutes retained after gates per scope, mean Noise coverage, which scans were produced, and presence-control parameters.

Summary. A lean *ingest* $\rightarrow$ *prepare* $\rightarrow$ *align* $\rightarrow$ *test* pipeline converts raw logs into minute tables and evaluates short *forward*-lag associations between speech-like activity and air metrics. *Robustness* comes from gap-aware gridding, light coverage gates, and presence-controlled residuals, with reverse-lag scans as diagnostics. *Reproducibility* is ensured by YAML configuration (window, weights). Optional complementary proxies can be enabled without altering the core flow.

4 Validation and Baseline

Bench and short pilot runs preceded the formal experiment to verify autonomous operation, fault handling, timing, storage behaviour, and mechanical robustness.

4.1 AirQ readiness

Tests. Each unit was provisioned via the setup script (Wi-Fi credentials provided), then power-cycled to confirm auto-start, and operated unattended for several days with periodic spot checks. Database records were inspected for schema compliance, units/ranges, timestamp continuity, and missing-data rate. **Result.** All AirQ devices self-configured correctly, auto-started reliably, logged continuously without intervention, and the data were ingested cleanly into the database with no corruption or schema drift.

4.2 Noise fallbacks

Tests. Failure handling and non-core features were exercised: microSD present/absent at boot, removal/reinsertion during operation, and index recovery after forced reset. Radio/time behaviour was stressed with Wi-Fi unavailability at boot and mid-run, using bounded NTP retries with backoff; logging was deferred until time was valid. Auxiliary functions were verified: MAX17048 battery readings and TFT display; button debounce with rising-edge trigger only; deferred backlight/display activation between acquisition cycles; BLE “peak” service (brief advertising on demand, remaining active if connected); and automatic deep sleep at critically low battery. **Result.** Behaviour matched the design—graceful degradation without data loss in baseline tests, preserved sampling cadence, and correct operation of all auxiliary features. A firmware bug discovered later during deployment was fixed. *Note:* logging was validated with Speed

Class 10, UHS-I, 32 GB microSD cards; equivalent behaviour is not guaranteed with other card types.

4.3 Cycle metrics (Noise kit)

A representative cadence with $f_s = 44.1$ kHz and $T_c \approx 1.5$ s was observed:

- Sampling window: $T_w = 500$ ms (22,050 samples).
- Processing: FFT ≈ 370 ms; logging 16–63 ms.
- **Active time:** ≈ 0.886 – 0.933 s per cycle (**59.1–62.2 %**).
- **Idle time:** ≈ 0.610 – 0.560 s per cycle (**40.9–37.8 %**).
- Frame size: 2048 bins, 16,896 B per frame (512 B aligned).

Voice detection. Presence was determined by the embedded pipeline. Because full spectra were logged, equivalent or alternative detectors can be applied *a posteriori* without firmware changes.

4.4 BLE peak path (Noise kit)

Test. A known single-tone stimulus was injected; the on-device FFT identified the dominant bin and published the peak via BLE. The simultaneously recorded spectrum was examined offline. **Result.** The advertised peak coincided with the recorded dominant frequency accounting for FFT resolution.

4.5 Battery autonomy (Noise kit)

Test. An endurance run from full charge was performed with the standard cadence and features. Figure 19 reports battery % (left axis) and voltage (right axis). **Result.** Starting at 100 % and 4.20 V, both trajectories declined steadily and reached the low-battery region near 3.63 V and < 5 % after ≈ 23.4 h. Practical autonomy under the current settings is therefore about **24 h**. Reducing duty cycle and/or payload would extend runtime, although aggressive thinning would undermine the aim (intermittent speech requires frequent observation).

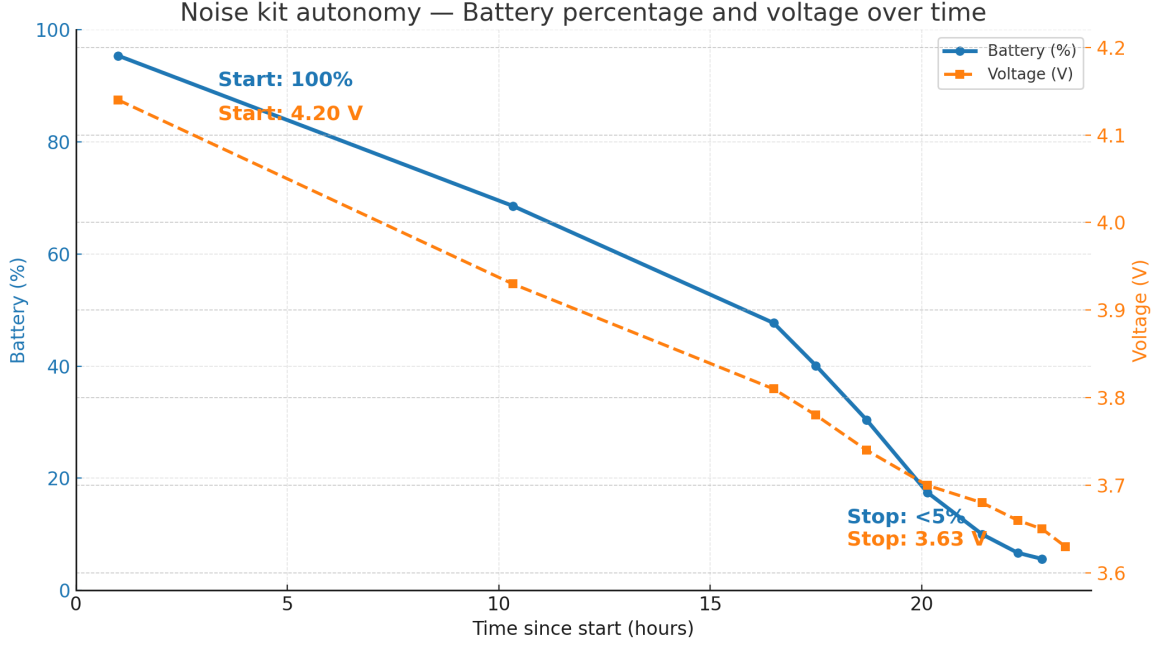


Figure 19. Noise kit autonomy with twin axes: battery percentage (left) and voltage (right) versus time since start (hours). Annotations mark the initial state (100 %, 4.20 V) and the stop condition (< 5 %, 3.63 V).

4.6 Mechanical robustness

Tests. Repeated assemblies were performed to assess fit, fastening, and access to internals. **Result.** Both enclosures were sturdy and precise. Minor gaps, when present, were insignificant; the halves remained securely mated yet easy to separate. Internals were held firmly while allowing removal/swapping when required. Fresh prints occasionally required light force for tight press-fits; after a few cycles, the interfaces settled into comfortable tolerance. One TFT PCB fractured at optional snap-tabs during a first assembly; subsequent assemblies succeeded using minimal tools (a screwdriver) and, for the SCD30 sensor, a short loop of thread to guide insertion into the bayonet socket. Overall, assembly/disassembly were straightforward and repeatable.

4.7 Analysis-software validation

Test. The data-analysis pipeline was exercised on artificially generated pairs designed to (i) exhibit near-perfect correlation at a known positive lag, and (ii) show no correlation. The correlated case used a known lag to emulate a causal offset. **Result.** The pipeline recovered the expected best lag, rejected H_0 in the correlated case, and did not reject H_0 for null pairs. Outputs matched the theoretical expectation, with a consistent shift attributable to the trailing-mean group delay in the derivative (the smoothing introduces a one-sample delay), which was accounted for in interpretation.

Readiness for deployment Collectively, these pre-deployment checks verified correct functionality, performance within the timing budget, and graceful fault handling; both AirQ and Noise kits—together with the analysis software—were cleared for mostly unattended experimental deployment.

5 Experimental Setup

5.1 Site

The deployment took place in a 22 m² ICT-company office in Reus with typical hardware use. Occupancy varied between 1–5 workers, with alternating quiet periods and single/group talking. Contextual sources included an intermittently operating air conditioner, a fan, office electronics, and a coffee machine. A window and the office door were occasionally opened, creating short ventilation events and introducing sporadic noise.

5.2 Layout/positions

Four units were installed and tagged for field identification. Table 8 summarises kit type, power mode, placement, and whether data from the unit were included in analysis. Figure 20 shows the device placements. Figure 21 compiles an overview photo of all kits used in the campaign alongside a phone screenshot confirming the BLE peak readout during commissioning.

Field tag	Kit	Power	Placement	In analysis
Air_001	AirQ	Mains	Centre of table cluster	Yes
Air_002	AirQ	Mains	Left side, near window	Yes
Mic_101	Noise	Battery	Right side, bookshelf near fan	No
Mic_102	Noise	Mains	Left side, near window	Yes

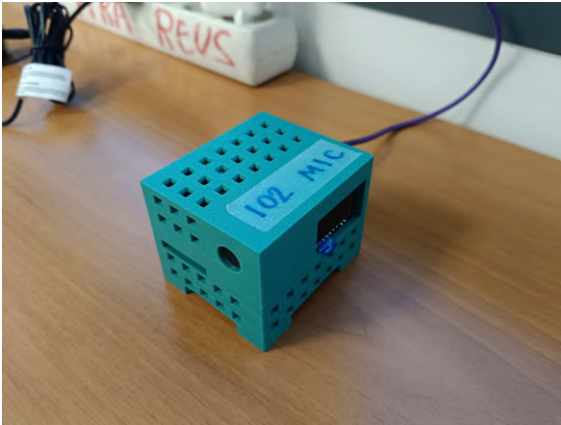
Table 8. Devices, power mode, placement, and inclusion.



(a) Air_001 — centre of table cluster.



(b) Air_002 — near window.

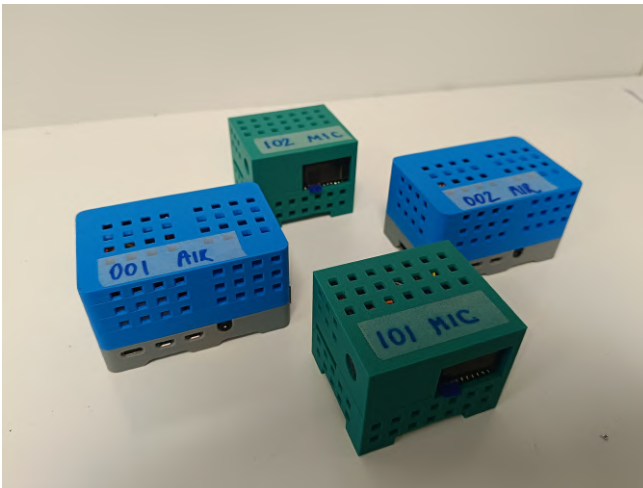


(c) Mic_102 — near window (mains).

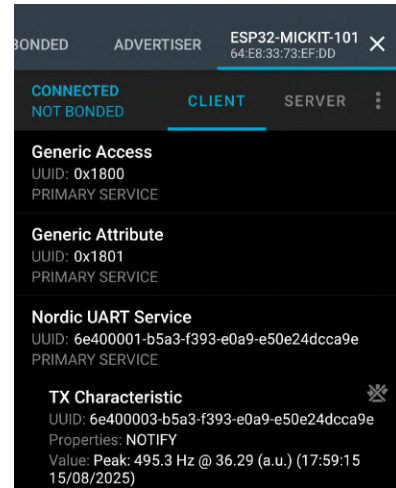


(d) Mic_101 — bookshelf near fan (battery).

Figure 20. Field deployment: device placements (tags as in Table 8).



(a) All kits used during the campaign.



(b) BLE peak readout.

Figure 21. Commissioning overview: full set of kits and Bluetooth verification screenshot.

5.3 Schedule

The final analysis window spans **12 Aug 2025, 10:15** to **14 Aug 2025, 18:00** (Europe/Madrid), covering three office days. Devices operated continuously within

this window at the site. **Mic_101** (battery) was removed at the end of the work day for recharging and returned the next morning; see deviations below.

5.4 Procedures

Commissioning checks

- **Power/health.** All four units booted correctly; SD cards reported healthy.
- **Networking/time.** Office Wi-Fi reachable; NTP time synchronisation confirmed on all devices at install.
- **BLE.** Advertising observed; correct data push verified using audio frequency generator; auto shutdown on advertising window expiry confirmed.
- **Spot checks.** Noise peak-frequency readout captured on phone; AirQ units verified via screen attach, with DB and acquisition scripts observed running during the campaign.

Deviations and exclusions

Mic_101 required overnight recharging and was taken off-site. With the office Wi-Fi SSID hardcoded, the unit could not complete NTP time synchronisation at home; to avoid off-context logging, the microSD was removed during charging. On return, this combination (no SD + no prior time synchronisation) tripped overly strict protection logic: the logger restarted from the beginning of the first data file and overwrote earlier data, including a portion of the records produced after a successful morning boot. Consequently, **Mic_101** is excluded from analysis. The remaining devices (**Mic_102**, **Air_001**, **Air_002**) behaved as expected and supply the analysis dataset. The incident exposed a software bug that was subsequently fixed.

Handoff to analysis

Data from **Air_001** and **Air_002** were exported from their on-device SQLite databases to CSV and merged into a single, kit-tagged SQLite database (.db). The unified database was cleaned and aggregated onto a *one-minute* grid prior to Parquet export. For sound, only **Mic_102** entered analysis: its spectrum frames were parsed from the binary logs using their index metadata, processed by the external voice-feature extractor, and aggregated to the same *one-minute* rows to produce a Parquet dataset. The AirQ and Noise Parquet datasets were then time-aligned and fed to the analysis pipeline to compute correlations and related statistics.

6 Results and Discussion

This section integrates results from the experimental deployment with their interpretation. All lags, statistics, and decisions arise from the minute-level pipeline after QC and gridding.

6.1 Decision rule and hypotheses

For each Air endpoint, the analysis scans short *forward* lags (Noise leading Air), corrects for autocorrelation via an effective sample size, and applies BH-FDR across lags

at $\alpha = 0.05$. One-sided alternatives are used for CO_2 , ΔCO_2 , and PM (expecting positive association); two-sided for TEMP and HUM. Negative-lag scans (Air leading Noise) are reported diagnostically only.

Endpoint alternatives. CO_2 : greater; ΔCO_2 (mean3): greater; $\text{PM}_{2.5}$: greater; PM_{10} : greater; TEMP: two-sided; HUM: two-sided.

Primary inference scope. To test *short-term* coupling between transient voice proxies and air responses, the **presence-controlled (occupied residuals)** scope is treated as **primary**. By removing slow presence (low-pass CO_2 , 30-min half-life) and the 24 h rhythm from *both* series, this scope down-weights shared drifts (e.g., occupancy buildup, HVAC scheduling, diurnal cycles) and preserves minute-scale departures. The ALL and OCCUPIED scopes are reported for ecological context and sensitivity.

6.2 Why emphasise the presence-controlled (occupied residuals) scope?

What it removes. Slow co-variation from presence and time-of-day can inflate short positive-lag correlations. Regressing a low-pass CO_2 proxy and diurnal sine/cosine terms targets those backgrounds while retaining minute-scale structure.

When it helps. For hypotheses about rapid effects (e.g., speech or activity leading to an air response within minutes), residualisation improves specificity by focusing on bursts rather than baselines.

Limits. It may attenuate genuine slower pathways (such as thermal drift), so results are interpreted alongside ALL and OCCUPIED scopes for robustness.

6.3 Pair-level summaries at a glance

Tables 9–10 compact the best-lag outcomes for each Air endpoint and scope. Cells show *lag/r*; ✓ marks tests significant after BH–FDR correction with the expected sign. **Scopes:** **ALL** (full window), **OCCUPIED** (minutes with presence), **PRESENCE-CTRL** (occupied residuals: slow presence and diurnal rhythms removed from both series; *primary* for short-lag inference).

Endpoint	ALL	OCCUPIED	PRESENCE-CTRL
CO_2	+10 / +0.114 ✓	+10 / +0.157 ✓	+4 / −0.012 ×
ΔCO_2 (mean3)	+2 / +0.172 ✓	+2 / +0.188 ✓	+4 / +0.096 ✓
$\text{PM}_{2.5}$	+8 / +0.060 ×	+8 / +0.030 ×	+8 / −0.001 ×
PM_{10}	+8 / +0.060 ×	+8 / +0.031 ×	+8 / +0.001 ×
TEMP	+10 / −0.179 ✓	+7 / −0.162 ✓	+10 / −0.018 ×
HUM	+10 / +0.083 ✓	+8 / +0.047 ×	+8 / +0.088 ✓

Table 9. AIRQ001 × NOISE102 — best-lag summary across scopes (experimental dataset). PRESENCE-CTRL is the *primary* scope for short-lag inference.

Endpoint	ALL	OCCUPIED	PRESENCE-CTRL
CO ₂	+10 / +0.121 ✓	+10 / +0.168 ✓	+3 / -0.016 ×
ΔCO ₂ (mean3)	+6 / +0.156 ✓	+6 / +0.155 ✓	+4 / +0.110 ✓
PM _{2.5}	+8 / +0.045 ×	0 / +0.017 ×	+8 / -0.009 ×
PM ₁₀	+8 / +0.044 ×	0 / +0.017 ×	+8 / -0.011 ×
TEMP	+2 / -0.185 ✓	+2 / -0.191 ✓	+10 / -0.091 ✓
HUM	+8 / +0.036 ×	0 / +0.011 ×	+8 / +0.045 ×

Table 10. AIRQ002 × NOISE102 — best-lag summary across scopes (experimental dataset). PRESENCE-CTRL is the *primary* scope for short-lag inference.

6.4 Layout-aware reading of the observed results

The analysed pairs reflect two geometries (Table 8, Figure 20): (i) *cross-room*—AIRQ001 (centre) with NOISE102 (left, near window); and (ii) *near-co-located*—AIRQ002 (left, near window) with NOISE102. Mic₁₀₁ (right/bookshelf near fan) was excluded.

AIRQ001 vs. NOISE102 (centre–window). *ALL/OCCUPIED*: CO₂ peaks at +10 min (small positive r); ΔCO₂ peaks at +2 min (small positive r). TEMP shows a short-lag negative correlation (significant); HUM is weak and scope-dependent. *PRESENCE-CTRL (primary)*: CO₂ becomes non-significant; ΔCO₂ shows a weak positive association at +4 min; TEMP and PM are non-significant; HUM shows a weak positive correlation.

AIRQ002 vs. NOISE102 (window zone). *ALL/OCCUPIED*: CO₂ again peaks at +10 min; ΔCO₂ shifts to +6 min, consistent with stronger perimeter exchange delaying the derivative response. TEMP is negative at +2 min. HUM and PM remain non-significant. *PRESENCE-CTRL (primary)*: CO₂ is non-significant; ΔCO₂ shows a weak positive association at +4 min; after residualisation, only the perimeter unit (AIRQ002) remains significant for TEMP, shifting to a longer +10 min lag (negative r), while the centre unit (AIRQ001) is no longer significant. PM and HUM remain non-significant.

Net view grounded in the deployment. Across both layouts, forward short positive associations appear mainly for ΔCO₂ at $\approx +2$ to $+6$ min in ALL/OCCUPIED, converging to $\approx +4$ min in PRESENCE-CTRL. CO₂ *levels* peak later (around +10 min) in ALL/OCCUPIED but are not robust under PRESENCE-CTRL. PM shows no clear short-lag signal. Geometry modulates timing (shorter near centre, longer at perimeter).

6.5 Hypothesis decisions (per pair, per scope)

AIRQ001 × NOISE102

- CO₂ (Alt >): *Reject* H_0 in ALL/OCCUPIED at +10 min; *Fail* in PRESENCE-CTRL.
- ΔCO₂ (Alt >): *Reject* H_0 in ALL/OCCUPIED at +2 min and in PRESENCE-CTRL at +4 min.
- PM_{2.5}/PM₁₀ (Alt >): *Fail* in all scopes.

- TEMP (two-sided): *Reject* H_0 (negative r) in ALL/OCCUPIED; *Fail* in PRESENCE-CTRL.
- HUM (two-sided): *Reject* H_0 (small positive r) in ALL and PRESENCE-CTRL; *Fail* in OCCUPIED.

AIRQ002 $\times$ NOISE102

- CO₂ (Alt >): *Reject* H_0 in ALL/OCCUPIED at +10 min; *Fail* in PRESENCE-CTRL.
- Δ CO₂ (Alt >): *Reject* H_0 at +6 min in ALL/OCCUPIED and at +4 min in PRESENCE-CTRL.
- PM_{2.5}/PM₁₀ (Alt >): *Fail* in all scopes.
- TEMP (two-sided): *Reject* H_0 (negative r) in ALL/OCCUPIED at +2 min and in PRESENCE-CTRL at +10 min.
- HUM (two-sided): *Fail* in all scopes.

6.6 Immediate interpretation of the gathered data

Under the **primary** scope (PRESENCE-CTRL), a small, consistent positive association appears between the voice proxy and Δ CO₂ at +4 **min** for both pairs, while CO₂ levels become non-significant. PM shows no robust short-lag coupling. *Temperature is location-dependent*: short-lag negatives in ALL/OCCUPIED; after residualisation, only the perimeter unit (AIRQ002) remains significant in PRESENCE-CTRL at a longer +10 min lag (negative r), whereas the centre unit (AIRQ001) is not significant. Humidity is weak/inconsistent (AIRQ001 significant in ALL and PRESENCE-CTRL only). In ALL/OCCUPIED the picture is similar but more exposed to shared slow trends. Effect sizes are modest ($r \approx 0.11$ – 0.19 ; $r^2 \leq 0.037$).

Correlation vs. causation (grounded in the experiment). Findings are correlational. Signs and lags are consistent with occupancy/speech contributing to local changes under the room’s airflow, HVAC, and solar load, but no causal claim is made. Reverse-direction checks (Air leading Noise, $-10 \dots -1$ min) are smaller or comparable in $|r|$ and do not overturn the forward-lag pattern.

Implications for Phase 2

Residual analyses suggest that minute-scale coupling is better represented by the *rate of change* rather than the absolute level. Accordingly, Phase 2 will emphasise:

1. prioritising Δ CO₂ (short rolling mean) as the primary short-lag proxy;
2. retaining minute-level aggregation with background removal applied;
3. favouring central placements, since perimeter conditions alter timing.

Given the modest effect sizes, relatively dense acoustic sampling should be preserved; the duty cycle may be reduced only if the voice proxy proves stable. PM will be retained mainly for contextual information and episodic events rather than short-lag coupling.

Limitations

A single room and limited window, two geometries, and modest effects ($r \approx 0.11$ – 0.19) limit generality. Spatial separation, mixing times, and ventilation heterogeneity affect lag interpretability. Residualisation improves specificity for bursts but may attenuate slower pathways. Voice proxies lack independent ground truth. Sensor/place-ment factors (optical PM interference, CO₂ mixing, self-heating, drafts) introduce non-stationarity. BH-FDR controls lags per endpoint, but selection over scopes/locations may still favour optimistic patterns. Minute gridding smooths sub-minute transients.

6.7 Operational notes on the kits

Analyses used a fixed minute grid per kit within the configured window, keeping gaps explicit. Time was normalised at ingestion (local→UTC, DST-safe). AirQ units and the Noise kit operated practically unattended; spectra and derived voice proxies were logged at the configured cadence and ingested without schema drift, supporting like-for-like comparisons and lag scans.

Key message. Across both geometries and all scopes, the only air endpoint that consistently shows a short *forward*-lag association with voice is the **rate of change of CO₂** (dc02_mean3). In the **presence-controlled** scope (primary), the association converges at +4 **min** for both pairs; in ALL/OCCUPIED it appears at +2 and +6 min depending on location. CO₂ levels, PM, and HUM are not consistently significant; TEMP is location/scope dependent (only AIRQ002 remains significant in PRESENCE-CTRL at +10 min).

6.8 Illustrative figure: NOISE102 voice timeline

Figure 22 provides a visual example of the minute-level voice metrics used in this study. The blue trace is `voice_rate_time`; the orange trace is the 5-minute rolling mean (moving average) of that proxy. Horizontal dashed lines mark reference thresholds. The secondary (right) axis overlays a mean-rise intensity proxy (dB scale) aligned in time. This plot summarises how continuous audio from NOISE102 was processed, aggregated, and visualised prior to the lagged correlation analysis.

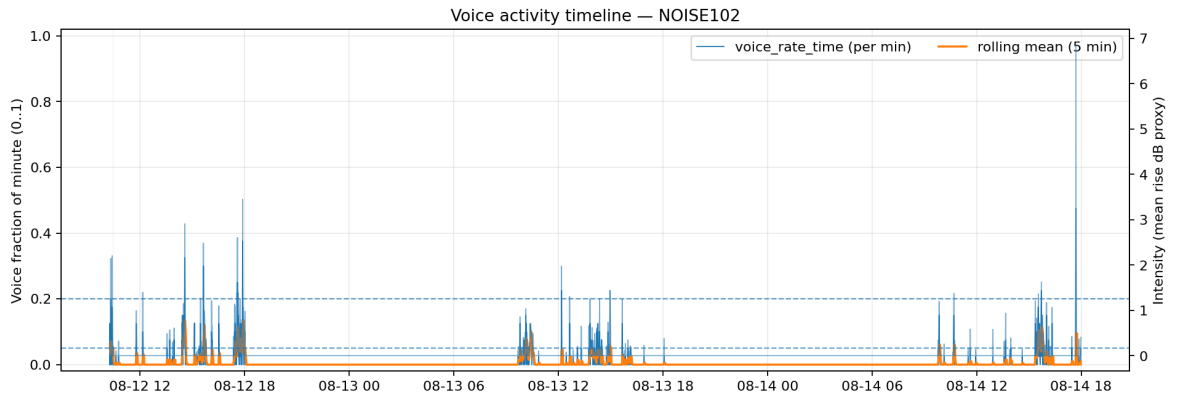


Figure 22. Voice activity timeline — NOISE102. Blue: `voice_rate_time`. Orange: 5-minute rolling mean. Dashed lines: reference thresholds. Right axis: mean-rise intensity (dB proxy). The figure illustrates the aggregation and processing pipeline that yields the minute-scale voice metrics used in the analysis.

7 Future Work

Baseline and motivation. Phase 1 established reliable, privacy-preserving capture: *AirQ* persists room-scale air metrics in local **SQLite**; *Noise* logs full spectra as 512 B-aligned **binary** .BIN frames (interleaved **float32** frequency–magnitude pairs) alongside derived metrics. Minute-level offline summaries now serve as ground truth for designing on-device proxies and a leaner Phase 2 data plane.

Observed bounds. Fixed cadences and rich acoustic logs raise storage and power budgets; per-device configuration is still required; remote health is basic; most interpretation is post hoc. On-device detection is intentionally minimal.

Phase 2: from stand-alone kits to a self-organising fleet

Coordination and data.

- **Self-organisation.** Monitoring Units (MUs) discover peers, exchange liveness, and elect lightweight roles (collector, sensor-only, relay) that adapt to joins, leaves, and faults.
- **Lean data plane.** A compact, versioned schema carries minute summaries by default; short *diagnostic bursts* (episodic raw windows) are pulled on triggers or for audits.

Sensing, storage, and analytics.

- **Operational vs. diagnostic.** Keep SQLite and full-spectrum binaries for commissioning and audits; routine operation emits summarised frames to reduce compute, I/O, and bandwidth.
- **Ephemeral ring buffers.** Maintain a short in-memory buffer; persist raw windows only on events, anomalies, or remote pulls.
- **Adaptive duty cycle.** Gate acquisition with a compact, room-agnostic detector aligned with the offline pipeline; raise cadence on useful signal, lower it otherwise.
- **Quality scaling (*Noise*).** If proxies remain reliable, progressively downscale capture (sample rate/bit depth, voice-band focus, FFT size/resolution, frame subsampling) to recover efficiency; roll back on degradation (guarded by continuous validation).
- **Optional AI assist.** Evaluate lightweight classifiers on spectral summaries; update via signed parameter/model packs when beneficial.

Operations and privacy.

- **Fleet management.** Signed parameter packs (sampling, gating, reporting), a typed error ledger, robust time handling, and per-unit diagnostics for remote inspection and maintenance.
- **Privacy by design.** No raw speech by default; buffers remain ephemeral; encryption *in transit and at rest* when required; identifiers scoped to a local site collector.

Expected outcomes and evaluation

- **Efficiency.** Fewer stored/transmitted bytes and lower energy per useful minute in routine operation.
- **Scalability.** Drop-in multi-room deployments with stable behaviour under joins/leaves.
- **Scientific leverage.** Minute summaries available online; episodic raw captures preserved for root-cause analysis and audits.
- **Validation.** Pilot studies with co-located pairs and annotated events, benchmarking on-device proxies (and quality-downscaled modes) against the established offline pipeline.

Outlook. A fleet of MUs that self-coordinates, speaks a compact dialect, samples adaptively (with quality scaled to need), and preserves auditability on demand—turning Phase 1 reliability into *operational capability* at scale.

8 Conclusions

What was delivered. Phase 1 produced two robust, modular kits and a reproducible pipeline. *AirQ* logs room-scale air metrics to SQLite. *Noise* acquires high-rate audio, computes spectral features, and writes compact, 512 B-aligned binary frames without storing intelligible speech. Purpose-built enclosures and fault-tolerant firmware/software enabled unattended operation and clean datasets. The implementation shows low technical debt: well-structured modules and a custom API ensure maintainability; all constants (“magic numbers”) are named and commented, yielding efficient, verifiable, and extensible code.

Engineering validation. Both kits start on power-up and run autonomously with bounded recovery from common faults. *AirQ*: supervised sensor processes, explicit time synchronisation at boot, and a stable SQLite schema; storage remains low (≈ 50 MB/month at 1-min cadence). *Noise*: timing budgets are sustained via PSRAM double buffering and aligned writes; power-loss safety and index recovery limit corruption risk; time-gated logging prevents misdated records; storage needs are characterised (≈ 0.9 GiB/day at current settings); battery autonomy ≈ 24 h. Enclosures are reproducible and mechanically sound under repeated assembly.

Empirical findings

Observed evidence (three deployment days). Minute-level forward-lag scans show small but consistent associations between speech-like activity and the *rate of change* of CO₂ (dCO_{2_3mean}). Under the *presence-controlled* scope—primary for short-lag inference—the signal concentrates near +4 minutes for both analysed pairs. CO₂ *levels* lose significance once backgrounds are removed; PM_{2.5}/PM₁₀ show no robust short-lag links; HUM is weak/inconsistent; TEMP remains location-dependent. Effect sizes are modest (Pearson $r \approx 0.11$ – 0.19); associations are reported, not causal effects.

Limitations. Single site and short window; no independent ground truth for voice presence; spatial separation and HVAC/mixing heterogeneity affect lag interpretation. Residualisation improves specificity for minute-scale bursts but may attenuate slower pathways. Commodity sensors introduce non-stationarity, constraining external validity.

Practical implications. For minute-scale coupling, *derivative* endpoints (short-horizon ΔCO_2) are more informative than levels. Dense acoustic sampling is essential—speech is intermittent—while air streams can remain at minute resolution. Logging the combined spectrum per frame preserves the ability to re-run alternative detectors *ex post* without firmware changes.

Reproducibility

Artefacts. All core artefacts—3D-print files, bills of materials, wiring maps, firmware, analysis code, and configuration files—are available in a public repository [68], organised for straightforward reproduction alongside this document.

Research dissemination. The following research publication has resulted from the study carried out in this work:

Edgar Batista, Constantin Shoot, Agustí Solanas, and Antoni Martínez-Ballesté, “*Self-Organising and Autonomous IoT-based Monitoring Units for Smart Environments*,” 13th International Conference on Applications in Electronics Pervading Industry, Environment and Society (ApplePies), Torino, Italy, 11–12 September 2025.

Future directions

Advance to a lightweight, *self-organising* multi-unit architecture with a compact, versioned data plane and adaptive duty cycles guided by a room-independent on-device detector. Fleet-level operation will rely on signed parameter packs, robust time management, and structured diagnostics—maintaining privacy, auditability, and reproducibility by default.

A Enclosure Prototyping and 3D-Print Iterations

This appendix shows the prototype-to-production path for the *AirQ* and *Noise* enclosures—from first fit checks to final batches—confirming CAD–hardware agreement and informing the final geometry.

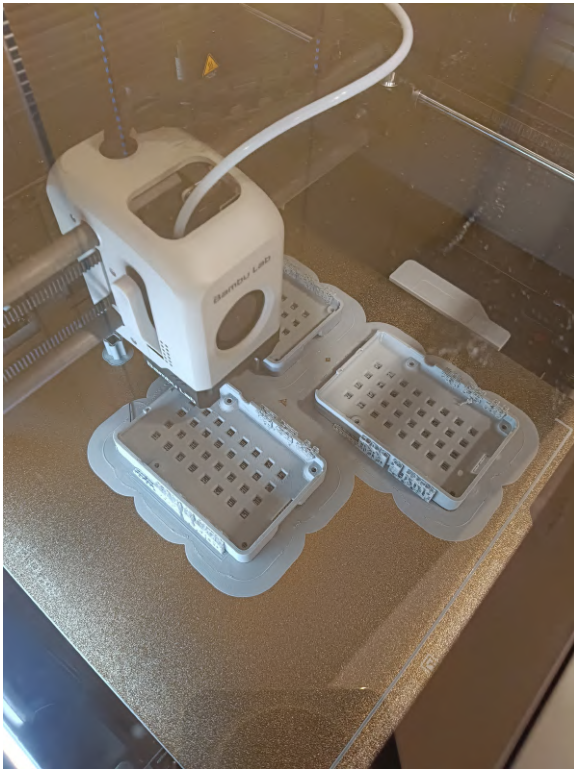


Side view of the first test print (modified base model).



Front view of the first test print (modified base model).

Figure 23. AirQ slightly modified base model, first verification print. *Left:* side view; *right:* front view. The Raspberry Pi fit confirmed that the original model required correction to align the USB-C charging port with the lower half. The print also validated dimensional correspondence between the Raspberry Pi 3D model and the real hardware, clearing it for continued use in shaping the enclosure.



AirQ bottom enclosure parts during the final print.

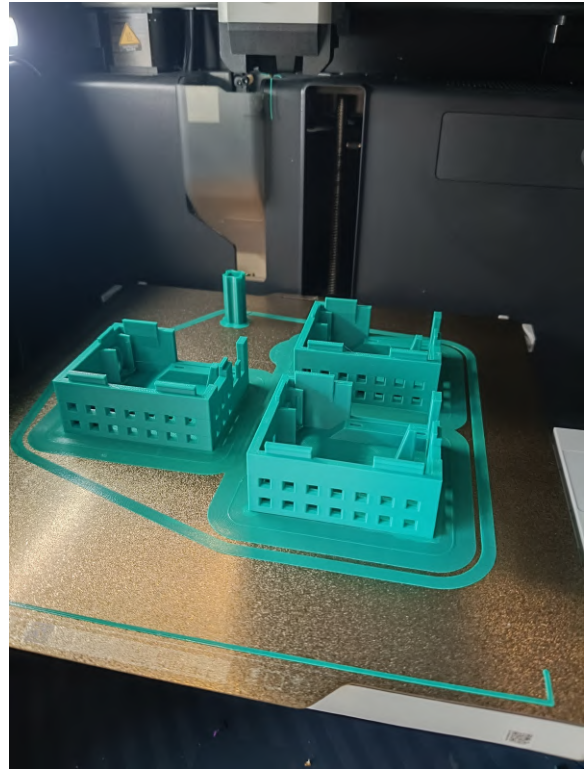


Finished batch of three AirQ enclosure bottom pieces.

Figure 24. AirQ enclosure bottom halves—final production. *Left:* bottom parts being printed; *right:* completed batch of three bottom parts.



Noise kit top parts printing inside printer.

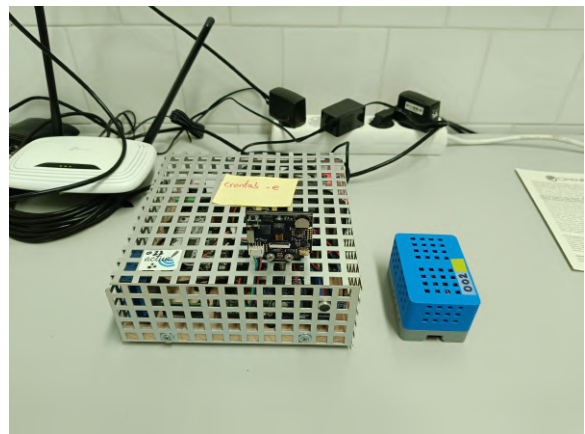


Close-up of the completed Noise top parts on the build plate.

Figure 25. Noise kit enclosure—top-piece production. *Left:* live print on a Bambu Lab P1S; *right:* close-up after completion, showing surface quality and dimensional features.



Noise (left) and AirQ (right) units side by side on the workbench.



AirQ next to an ACTUA-kit unit, showing size contrast and grille-inspired styling.

Figure 26. Form-factor comparisons. *Left:* Noise and AirQ together (antenna slit and microphone aperture on the Noise enclosure; microSD slit on the AirQ enclosure); *right:* AirQ beside an ACTUA project kit, highlighting the size difference and the grille-inspired aesthetic.



All produced kits grouped for final inspection. Selected failed and test prints from both AirQ and Noise iterations.

Figure 27. Production overview and iterations. *Left:* the complete set of fabricated units; *right:* representative failed/test pieces illustrating the trial-and-error refinements that led to the definitive enclosure parts for both kits.

B Budget and Procurement Summary

This appendix summarises the expenses for parts acquired specifically for this project (reused items from prior builds are excluded). Procurement prioritised reusing available components and purchasing only what was strictly necessary. The original PLA reels later showed moisture-related degradation (brittle, spaghetti-like breaks and nozzle clogs)—unexpected for a well-rated product—likely progressing after the factory seal was opened. By contrast, an alternative PLA brand eventually used for the *Noise* enclosures, stored for months outside its hermetic bag, remained dry and printed reliably.

BricoGeek				PcComponentes			
Product	Qty	PPU	Total	Product	Qty	PPU	Total
LiPo 3500 mAh	3	€12.50	€37.50	PLA Blue	1	€20.90	€20.90
ESP32-S3	3	€20.50	€61.50	PLA Gray	1	€20.99	€20.99
TFT LCD	3	€9.50	€28.50	Shipping etc.	1	€4.50	€4.50
DHT22	2	€3.95	€7.90	Subtotal			€46.39
Push Button	4	€0.35	€1.40				
Shipping etc.	1	€0.00	€0.00				
Subtotal			€136.80				

Table 11. Purchased components (excluding recycled parts). Column widths and alignment are adjusted to avoid overlap and place the second table flush right.

Grand total (BricoGeek + PcComponentes)	€183.19
---	---------

Table 12. Overall expenditure for purchased items.

References

- [1] Henk W. Brink, Marcel G. L. C. Loomans, Mark P. Mobach, and Helianthe S. M. Kort. “Classrooms’ Indoor Environmental Conditions Affecting the Academic Achievement of Students and Teachers in Higher Education: A Systematic Literature Review”. In: *Indoor Air* 31.2 (2021), pp. 405–425.
- [2] Pierfrancesco Bellini et al. “IoT-Enabled Smart Cities: A Review of Concepts, Frameworks and Key Technologies”. In: *Applied Sciences* 12.3 (2022), p. 1607.
- [3] Daniele Spoladore et al. “A Semantic-Enabled Smart Home for AAL and Continuity of Care”. In: *IoT in Healthcare and Ambient Assisted Living*. Vol. 933. 2021, pp. 343–371.
- [4] Antoni Martínez-Ballesté et al. “A Proposal for the Smart Classroom Infrastructure Using IoT and Artificial Intelligence”. In: *Proceedings of the 48th IEEE Annual Computers, Software, and Applications Conference (COMPSAC)*. 2024, pp. 109–114.
- [5] Ejaz Ahmed et al. “Internet-of-things-based smart environments: state of the art, taxonomy, and open research challenges”. In: *IEEE Wireless Communications* 23.5 (2016), pp. 10–16.
- [6] Juvenal Machin et al. “Multi-agent Systems for Pervasive Electronics: A Case Study in School Classrooms”. In: *Applications in Electronics Pervading Industry, Environment and Society*. Vol. 1110. 2023, pp. 474–479.
- [7] Edgar Batista et al. “Towards Context-Aware Classrooms: Lessons Learnt from the ACTUA Project”. In: *Applications in Electronics Pervading Industry, Environment and Society*. Vol. 1110. 2023, pp. 399–404.
- [8] URL: <https://www.espressif.com/en/products/socs/esp32-s3> (visited on 09/01/2025).
- [9] URL: <https://www.freertos.org/Why-FreeRTOS/What-is-FreeRTOS> (visited on 09/01/2025).
- [10] URL: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/system/freertos_idf.html (visited on 09/01/2025).
- [11] URL: <https://kwahe.medium.com/the-first-bug-on-mars-os-scheduling-priority-inversion-and-the-mars-pathfinder-53586a631525> (visited on 09/01/2025).
- [12] URL: <https://github.com/espressif/arduino-esp32> (visited on 09/01/2025).
- [13] URL: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/system/external_ram.html (visited on 09/01/2025).
- [14] URL: <https://docs.espressif.com/projects/esp-idf/en/v4.4/esp32s3/api-reference/peripherals/adc.html> (visited on 09/01/2025).
- [15] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-Time Signal Processing*. Prentice Hall, 1999.
- [16] URL: https://www.ni.com/docs/en-US/bundle/diadem/page/genmaths/genmaths/calc_fouriertransform.htm (visited on 09/01/2025).
- [17] URL: <https://brianmcfree.net/dstbook-site/content/ch06-dft-properties/Leakage.html> (visited on 09/01/2025).
- [18] Gunnar Fant. *Speech Acoustics and Phonetics*. Kluwer Academic Publishers, 1995.
- [19] URL: <https://www.modelo.io/damf/article/2024/04/24/1502/understanding-3mf-and-gcode-for-3d-printing?hl=en> (visited on 09/01/2025).
- [20] URL: <https://www.simplify3d.com/resources/materials-guide/pla/> (visited on 09/01/2025).
- [21] URL: <https://www.statisticssolutions.com/free-resources/directory-of-statistical-analyses/correlation-pearson-kendall-spearman/> (visited on 09/01/2025).
- [22] URL: <https://www.nonlinear.com/support/progenesis/comet/faq/v2.0/pq-values.aspx> (visited on 09/01/2025).
- [23] Germaine Cornelissen. “Cosinor-based rhythmometry”. In: *Theoretical Biology and Medical Modelling* 11.16 (2014), p. 16.
- [24] URL: <https://diaridigital.urv.cat/un-estudi-de-la-urv-monitorea-les-aules-per-avaluar-la-transmissio-de-virus-entre-infants/> (visited on 09/01/2025).
- [25] URL: <https://pinout.xyz/> (visited on 09/01/2025).
- [26] URL: https://pinout.xyz/pinout/enviro_plus (visited on 09/01/2025).
- [27] URL: https://cdn.shopify.com/s/files/1/0174/1800/files/enviro_plus_schematic.pdf?v=1666695786 (visited on 09/01/2025).

-
- [28] URL: https://www.aqmd.gov/docs/default-source/aq-spec/resources-page/plantower-pms5003-manual_v2-3.pdf (visited on 09/01/2025).
- [29] URL: <https://components101.com/sensors/dht22-pinout-specs-datasheet> (visited on 09/01/2025).
- [30] URL: https://sensirion.com/media/documents/4EAF6AF8/61652C3C/Sensirion_CO2_Sensors_SCD30_Datasheet.pdf (visited on 09/01/2025).
- [31] URL: <https://forum.fritzing.org/t/looking-for-pms5003-and-mp503-voc-libraries/18669> (visited on 09/01/2025).
- [32] URL: https://github.com/alfkoehn/CO2_monitor/blob/master/fritzing/SCD30.fpz (visited on 09/01/2025).
- [33] URL: <https://github.com/pimoroni/enviroplus-python> (visited on 09/01/2025).
- [34] URL: <https://forum.fritzing.org/t/dht22-library-as-pictured/15582/2> (visited on 09/01/2025).
- [35] URL: <https://cdn.shopify.com/s/files/1/0174/1800/files/bst-bme280-ds002.pdf?v=1662743150> (visited on 09/01/2025).
- [36] URL: https://optoelectronics.liteon.com/upload/download/DS86-2013-0003/LTR-559ALS-01_DS_V1.pdf (visited on 09/01/2025).
- [37] Used with explicit author permission; original license: CULTS PU / NO AI. URL: <https://cults3d.com/:1937761> (visited on 09/01/2025).
- [38] URL: <https://grabcad.com/library/raspberry-pi-4-model-b-1> (visited on 09/01/2025).
- [39] URL: <https://grabcad.com/library/dht22-module-1> (visited on 09/01/2025).
- [40] URL: <https://grabcad.com/library/pimoroni-pirate-audio-speaker-for-raspberry-pi-1> (visited on 09/01/2025).
- [41] URL: <https://grabcad.com/library/air-quality-sensor-pms5003-1> (visited on 09/01/2025).
- [42] URL: https://sensirion.com/media/documents/260AFF2D/616531AE/Sensirion_CO2_Sensors_SCD4x_STEP_file.step (visited on 09/01/2025).
- [43] URL: <https://github.com/pimoroni/bme280-python> (visited on 09/01/2025).
- [44] URL: <https://github.com/pimoroni/ltr559-python> (visited on 09/01/2025).
- [45] URL: <https://github.com/pimoroni/pms5003-python> (visited on 09/01/2025).
- [46] URL: <https://github.com/Sensirion/python-i2c-scd30> (visited on 09/01/2025).
- [47] URL: https://github.com/adafruit/Adafruit_CircuitPython_DHT (visited on 09/01/2025).
- [48] URL: https://cdn-shop.adafruit.com/product-files/5477/esp32-s3_datasheet_en.pdf (visited on 09/01/2025).
- [49] URL: <https://learn.adafruit.com/adafruit-esp32-s3-feather/pinouts> (visited on 09/01/2025).
- [50] URL: <https://cdn-shop.adafruit.com/datasheets/MAX9814.pdf> (visited on 09/01/2025).
- [51] URL: <https://cdn-shop.adafruit.com/datasheets/CMA-4544PF-W.pdf> (visited on 09/01/2025).
- [52] URL: <https://learn.adafruit.com/adafruit-mini-tft-0-dot-96-inch-180x60-breakout/pinouts> (visited on 09/01/2025).
- [53] URL: <https://ww1.microchip.com/downloads/en/DeviceDoc/MCP73831-Family-Data-Sheet-DS20001984H.pdf> (visited on 09/01/2025).
- [54] URL: https://cdn-learn.adafruit.com/assets/assets/000/110/822/original/adafruit_products_FESP3_sch.png?1649969039 (visited on 09/01/2025).
- [55] URL: <https://github.com/adafruit/Fritzing-Library> (visited on 09/01/2025).
- [56] URL: <https://grabcad.com/library/max9814-microphone-module-1> (visited on 09/01/2025).
- [57] URL: <https://grabcad.com/library/adafruit-feather-huzzah-1> (visited on 09/01/2025).
- [58] URL: <https://grabcad.com/library/adafruit-oled-breakout-board-16-bit-color-0-96-wmicrosd-holder-1> (visited on 09/01/2025).
- [59] URL: <https://grabcad.com/library/tactile-switch-short-1> (visited on 09/01/2025).
- [60] URL: <https://learn.adafruit.com/adafruit-esp32-s3-feather/arduino-ide-setup> (visited on 09/01/2025).

- [61] URL: https://github.com/espressif/esp-idf/blob/v5.4.2/examples/peripherals/adc/continuous_read/main/continuous_read_main.c (visited on 09/01/2025).
- [62] URL: <https://github.com/kosme/arduinoFFT> (visited on 09/01/2025).
- [63] URL: <https://github.com/adafruit/Adafruit-GFX-Library> (visited on 09/01/2025).
- [64] URL: <https://github.com/adafruit/Adafruit-ST7735-Library> (visited on 09/01/2025).
- [65] URL: https://github.com/adafruit/Adafruit_MAX1704X (visited on 09/01/2025).
- [66] URL: <https://github.com/arduino-libraries/SD> (visited on 09/01/2025).
- [67] URL: <https://github.com/adafruit/SdFat> (visited on 09/01/2025).
- [68] URL: https://github.com/pozdezh/Project_DELTA_Ph1 (visited on 09/02/2025).

